FPT UNIVERSITY

BÁO CÁO PHÂN TÍCH THỊ TRƯỜNG VÀNG VIỆT NAM

Group 1

Lecturer: Đào Thị Thanh

Github: https://github.com/WuWuyn/vn-gold-market-analysis

Tóm tắt điều hành

- Báo cáo chỉ sử dụng snapshot dữ liệu hiện có, mới nhất đến 2026-07-11. Kết luận không phải tín hiệu live sau ngày 2026-07-11.
- Khuyến nghị chính: có thể mua tích lũy có kiểm soát cho horizon khoảng 5 tháng, nhưng không mua mạnh một lần. Horizon 3 tháng không đủ hấp dẫn vì backtest tín hiệu âm; horizon 1 tháng chỉ phù hợp giao dịch nhỏ do tín hiệu lịch sử rất hiếm. Lưu ý: nếu mô hình snapshot 3 tháng bật tín hiệu, vẫn cần hạ trọng số nếu backtest 3 tháng có avg buy-day return âm.
- Target chính là lợi nhuận sau spread: mua tại giá bán hôm nay và bán lại theo giá mua ở ngày khả dụng gần nhất sau mốc 1/3/5 tháng lịch.
- Model frame có 5,485 ngày từ 2011-07-06 đến 2026-07-11; premium missing 0.0%.
- LightGBM, XGBoost và CatBoost được cài và train thật khi import được; deep models chỉ xuất hiện khi có runner huấn luyện thật.

Bản báo cáo này được viết theo hướng ra quyết định đầu tư chứ không chỉ mô tả biến động giá. Câu hỏi trung tâm là: nếu một nhà đầu tư cá nhân mua vàng SJC tại thời điểm snapshot, chịu giá bán ra hiện tại và có thể bán lại theo giá mua vào sau 1, 3 hoặc 5 tháng, lợi suất kỳ vọng có đủ bù cho spread, rủi ro giảm giá và chi phí cơ hội hay không. Vì vậy, kết luận mua/không mua được đọc cùng với xác suất lợi suất dương, phân vị rủi ro q10 và kết quả kiểm thử lùi cuốn chiếu.

Kết quả tại snapshot nghiêng về cách tiếp cận thận trọng: horizon 1 tháng, 3 tháng và 5 tháng được đọc riêng theo expected return, q10 downside và consistency backtest. Tín hiệu 5 tháng nếu có không đồng nghĩa với mua mạnh ngay lập tức, vì premium SJC và spread có thể đảo chiều nhanh khi NHNN thay đổi chính sách cung vàng hoặc thị trường quốc tế đổi chế độ lãi suất/USD.

Điểm cải thiện quan trọng so với bản caveat trước là premium coverage đã được rebuild theo hierarchy có gắn nhãn nguồn và proxy, giúp premium missing giảm xuống mức được báo cáo trong model frame. Dù vậy, lãi suất tiền gửi Việt Nam mới chỉ có dữ liệu forward-monitoring từ nguồn ngân hàng thương mại, còn news real-time availability vẫn chưa đủ mạnh để coi headline là tín hiệu giao dịch lịch sử tuyệt đối.

1. Business Understanding và góc nhìn kinh tế

Bài toán không chỉ là dự báo giá vàng, mà là quyết định có nên mua vàng vật chất tại Việt Nam trong 1-5 tháng tới. Người mua trả giá bán ra hôm nay và nếu thoát vị thế sẽ nhận giá mua vào trong tương lai, nên bid-ask spread và premium nội địa là một phần trực tiếp của lợi nhuận.

Vàng Việt Nam chịu tác động của bốn nhóm lực: giá vàng quốc tế, tỷ giá USD/VND, premium SJC do cấu trúc cung-cầu và chính sách, và chế độ rủi ro như lạm phát, lãi suất, địa chính trị, mùa vụ Tết/Thần Tài.

1.1. Từ bài toán giá sang bài toán quyết định

Nếu chỉ dự báo mức giá bán ra trong tương lai, mô hình rất dễ tạo ra tín hiệu đẹp trên giấy nhưng không dùng được ngoài thực tế. Nhà đầu tư vàng vật chất không mua ở mid-price và cũng không bán ở mid-price; họ mua theo giá bán ra của tiệm vàng/ngân hàng vàng và bán lại theo giá mua vào. Khoảng cách này chính là spread, và trong các giai đoạn căng thẳng spread có thể lớn đến mức ăn mòn toàn bộ lợi nhuận kỳ vọng.

Vì lý do đó, biến mục tiêu của đề tài được định nghĩa là lợi suất ròng theo horizon: giá mua vào tương lai chia cho giá bán ra hiện tại trừ 1. Định nghĩa này buộc mô hình phải trả lời đúng câu hỏi nghiệp vụ: người mua hôm nay có còn lời sau khi chịu chi phí giao dịch hay không. Đây cũng là lý do báo cáo không đưa ra một con số giá mục tiêu duy nhất, mà trình bày expected return, q10 downside và xác suất return dương cho từng chân trời.

1.2. Góc nhìn kinh tế - tài chính của thị trường vàng Việt Nam

Về mặt kinh tế, vàng là tài sản không tạo dòng tiền định kỳ. Khi lãi suất tiền gửi cao, chi phí cơ hội của việc nắm giữ vàng tăng; khi lãi suất thực giảm, chi phí cơ hội giảm và nhu cầu tích trữ vàng thường mạnh hơn. Trong bối cảnh Việt Nam, kênh vàng cạnh tranh trực tiếp với tiền gửi ngân hàng, bất động sản, chứng khoán và ngoại tệ, nên mô hình cần xem xét cả biến vĩ mô lẫn yếu tố thanh khoản nội địa.

Về mặt tài chính quốc tế, vàng thường được xem là tài sản phòng vệ trước lạm phát, rủi ro địa chính trị và bất ổn thị trường. Tuy nhiên, giá vàng trong nước không phải bản sao một-một của giá vàng thế giới. Khi nguồn cung vàng miếng SJC bị hạn chế hoặc cơ chế đấu thầu/chính sách thay đổi, premium nội địa có thể mở rộng hoặc thu hẹp độc lập với xu hướng toàn cầu.

Do đó, cách hiểu hợp lý là phân rã giá vàng Việt Nam thành ba lớp: lớp fair value toàn cầu quy đổi theo USD/VND, lớp premium thể chế - cung cầu trong nước, và lớp chi phí giao dịch/thanh khoản. Một quyết định mua tốt cần cả ba lớp cùng hỗ trợ: giá thế giới không quá bất lợi, tỷ giá không tạo rủi ro ngược, premium không quá đắt, và spread không làm méo lợi suất thực nhận.

Yếu tố	Cơ chế tác động	Biến dữ liệu dùng trong báo cáo
Giá vàng thế giới	Neo giá cơ bản theo USD/oz, phản ánh lãi suất thực, USD và safe-haven demand.	GC=F/LBMA proxy, global_gold_vnd_per_luong
USD/VND	Tăng USD/VND làm vàng quy đổi VND cao hơn ngay cả khi USD gold đi ngang.	usd_vnd_mid, usd_vnd_market_rate
Premium SJC	Chênh lệch nội địa phản ánh khan hiếm, quy định, đấu thầu và tâm lý tích trữ.	premium, premium_pct, source_dispersion
Thanh khoản/spread	Spread cao làm giảm lợi nhuận thực nhận và báo hiệu stress bán lẻ.	spread_abs, spread_pct
Rủi ro và mùa vụ	Crisis/policy/Tết/Thần Tài có thể làm premium và volatility tăng ngắn hạn.	event_regime, GPRD, raw_news intensity

2. Literature Review và lựa chọn phương pháp

Tài liệu dự báo vàng thường chia thành ba nhánh: mô hình chuỗi thời gian cổ điển, mô hình tài chính-vĩ mô có biến ngoại sinh, và mô hình machine learning/deep learning cho quan hệ phi tuyến. Với vàng Việt Nam, báo cáo ưu tiên mô hình có thể xử lý premium và biến ngoại sinh theo thời điểm công bố.

Nhóm mô hình chuỗi thời gian như ARIMA/SARIMAX phù hợp để tạo baseline vì giá vàng có tự tương quan, xu hướng và chu kỳ ngắn hạn. SARIMAX đặc biệt hữu ích khi muốn đưa thêm biến ngoại sinh như USD/VND, VIX, DXY, dầu, bạc hoặc chỉ báo rủi ro địa chính trị. Điểm mạnh của nhóm này là dễ kiểm soát leakage và dễ diễn giải; điểm yếu là khó bắt quan hệ phi tuyến giữa premium, sự kiện chính sách và tâm lý thị trường.

Nhóm kinh tế lượng đa biến như VECM/cointegration được xét vì giá vàng nội địa, giá vàng quốc tế quy đổi VND và tỷ giá có thể có quan hệ cân bằng dài hạn. Nếu tồn tại cointegration ổn định, sai lệch giữa giá nội địa và fair value toàn cầu có thể chứa thông tin mean-reversion. Tuy nhiên, kết quả screen hiện tại chưa đủ điều kiện để promote VECM thành mô hình forecast chính, nên báo cáo ghi nhận như một kiểm tra cấu trúc thay vì ép dùng.

Nhóm machine learning như Random Forest, Gradient Boosting, LightGBM, XGBoost và CatBoost được dùng để kiểm tra liệu quan hệ phi tuyến có cải thiện dự báo hay không. Với thị trường vàng Việt Nam, đây là nhóm đáng thử vì tác động của premium thường theo regime: premium thấp có thể là cơ hội, nhưng premium quá cao lại có thể là rủi ro đảo chiều nếu nguồn cung chính sách xuất hiện. Tree boosting có khả năng bắt các ngưỡng này tốt hơn hồi quy tuyến tính.

Báo cáo cũng phân biệt rõ mô hình dự báo kỳ vọng và mô hình dự báo rủi ro. Một expected return dương không đủ để mua nếu phân vị q10 quá âm, vì nhà đầu tư cá nhân chịu drawdown thực và chi phí thoát vị thế. Do đó, hồi quy phân vị/quantile boosting được dùng để ước lượng downside, còn rule quyết định cần đồng thời thỏa xác suất return dương và sàn q10.

Nhóm phương pháp	Vai trò trong bài toán vàng	Cách áp dụng trong dự án
ARIMA/SARIMAX	Baseline mạnh cho chuỗi có tự tương quan và biến ngoại sinh.	SARIMAX+exog trên target return nhiều horizon.
VECM/Cointegration	Phù hợp khi giá nội địa, vàng thế giới và FX có quan hệ cân bằng dài hạn.	VECM screen được ghi nhận; forecast chưa promote trong runner v1.
GARCH/volatility	Đánh giá rủi ro, volatility clustering và stress regime.	Dùng trong literature/methodology; volatility rolling trong EDA.
Tree boosting / Random Forest	Bắt tương tác phi tuyến giữa premium, events, FX, GPR.	Random Forest mean model; Gradient Boosting quantile cho downside q10.
Quantile regression	Quyết định mua cần downside risk, không chỉ expected return.	q10 forecast kết hợp với xác suất return dương.
DeepAR/TFT/N-BEATS/N-HiTS	Ứng viên multi-horizon probabilistic forecast khi có panel đủ giàu.	Ghi là hướng tiếp theo; không train nếu thiếu dependency.

Lý do không chọn một mô hình duy nhất ngay từ đầu: thị trường vàng vừa có thành phần xu hướng toàn cầu, vừa có premium nội địa phi tuyến và event-driven. Vì vậy đánh giá theo walk-forward và quyết định dựa trên cả MAE, directional accuracy, q10 downside và signal performance.

Một điểm quan trọng của literature review là không xem mô hình phức tạp hơn là mặc nhiên tốt hơn. Nếu boosting hoặc deep learning chỉ cải thiện sai số trung bình nhưng làm tín hiệu giao dịch kém ổn định, mô hình đó không được ưu tiên cho quyết định mua vàng vật chất. Ngược lại, một baseline đơn giản nhưng có directional accuracy ổn định và downside có kiểm soát có thể hữu ích hơn cho nhà đầu tư.

3. Data Requirements, Collection và Data Understanding

Yêu cầu dữ liệu xuất phát từ phương trình kinh tế: giá vàng nội địa xấp xỉ giá vàng thế giới quy đổi VND cộng premium nội địa, sau đó bị điều chỉnh bởi spread, mùa vụ, chính sách và rủi ro. Vì vậy data lake phải có target nội địa, reference toàn cầu, FX, macro, event và kiểm soát thời điểm công bố.

Bảng kiểm kê dưới đây không chỉ liệt kê file, mà thể hiện vai trò của từng lớp dữ liệu trong pipeline. Nhóm target nội địa cung cấp nhãn huấn luyện và giá thực thi; nhóm global/reference tạo fair value để tính premium; nhóm FX chuyển USD gold sang VND/lượng; nhóm macro/news/event mô tả điều kiện thị trường; còn nhóm modeling lưu frame cuối, prediction và decision signal để có thể đối chiếu ngược mọi con số trong báo cáo.

Yêu cầu quan trọng nhất là tính lịch sử hợp lệ. Một nguồn chỉ được dùng làm nhãn nếu có thể chứng minh dữ liệu trả về đúng ngày nghiệp vụ đã yêu cầu, không phải giá hiện tại bị leak vào quá khứ. Nguyên tắc này làm pipeline chậm và nghiêm ngặt hơn, nhưng cần thiết vì một sai lệch current-leak nhỏ cũng có thể làm backtest vàng trông tốt giả tạo.

Dataset	Rows	Cols	Thời gian	Vai trò
SJC historical target	28,230	13	2011-07-06 đến 2026-07-11 (date)	Nguồn target huấn luyện chính; giữ bản ghi historical-valid.
Domestic daily panel	111,942	18	2010-01-02 đến 2026-07-11 (date)	Panel giá vàng trong nước nhiều nguồn.
Global reference daily	4,356	17	2010-01-01 đến 2026-07-10 (date)	Biến toàn cầu và FX dùng để giải thích giá vàng nội địa.
Premium enriched	5,485	24	2011-07-06 đến 2026-07-11 (date)	Phân rã premium SJC so với vàng thế giới quy đổi VND/lượng.
VN macro as-of	36,705	10	2010-01-01 đến 2026-07-11 (available_from)	Panel vĩ mô với mốc công bố để chống leakage.
GPR daily	15,163	9	1985-01-01 đến 2026-07-07 (date)	Chỉ số rủi ro địa chính trị.

Event regime	1,850	13	2010-04-15 đến 2027-10-05 (event_date)	Tết, Thần Tài, mùa cưới, chính sách, crisis/regime events.
News raw headlines	3,441	16	2010-11-01 đến 2026-07-11 (event_date)	Nguồn headline/backfill cho biến intensity và sentiment heuristic.
Model frame	5,485	783	2011-07-06 đến 2026-07-11 (date)	Bảng cuối cho huấn luyện: một dòng mỗi ngày, target 1/3/5 tháng lịch.
Model results	186	8	Không có cột ngày chuẩn	Leaderboard walk-forward theo horizon và fold.
Walk-forward predictions	56,085	8	2023-01-01 đến 2026-06-11 (date)	Dự báo từng fold để audit model behavior.
Decision signals	27,585	15	2023-01-01 đến 2026-04-11 (date)	Bảng tín hiệu mua theo threshold grid.
Trading signals	3,065	17	2023-01-01 đến 2026-04-11 (date)	Bảng collapse một dòng/ngày cho paper trading.

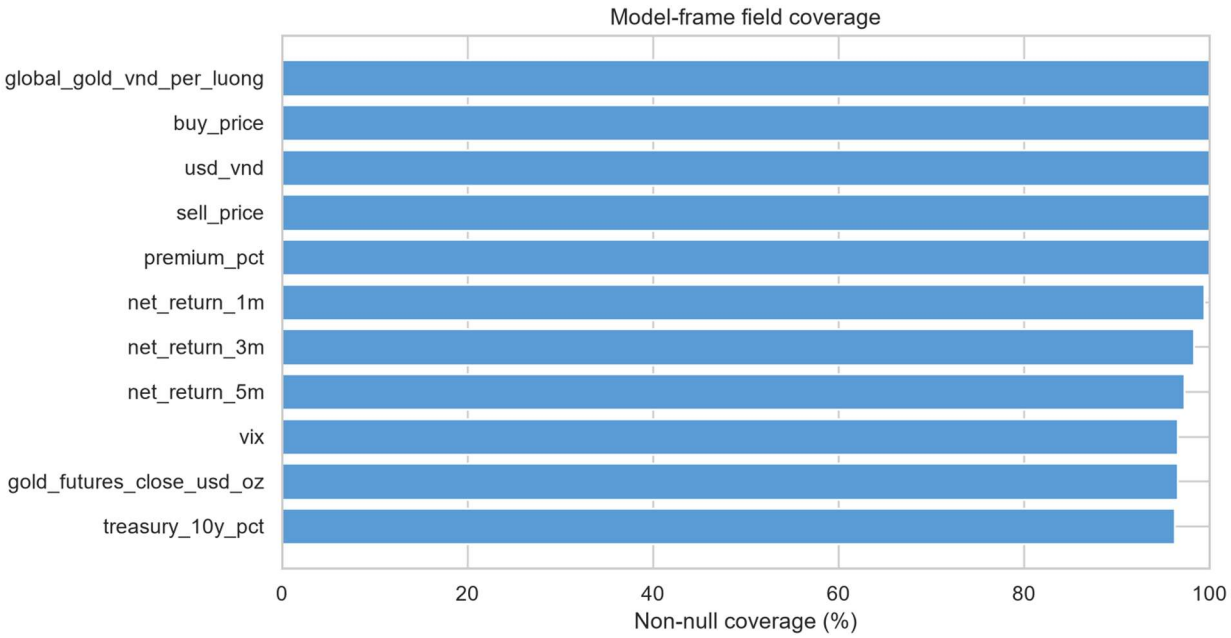
3.1. Data dictionary và khái niệm trường dữ liệu

Phần này đặt định nghĩa biến ngay cạnh kiểm kê dữ liệu để người đọc hiểu một cột trong CSV đang đại diện cho đại lượng kinh tế nào. Với vàng Việt Nam, cùng là 'giá vàng' nhưng nếu không phân biệt giá mua, giá bán, fair value toàn cầu, premium và spread thì mô hình rất dễ học nhầm tín hiệu.

Nhóm biến	Trường chính	Đơn vị / grain	Cách đọc kinh tế	Rủi ro dữ liệu
Domestic SJC target	buy_price, sell_price, mid_price, spread_pct	VND/lượng, một dòng mỗi ngày	Giá bán là chi phí vào lệnh; giá mua tương lai là giá thoát vị thế. Spread là chi phí giao dịch thực tế.	Raw crawl có mixed-grain, giá 0 hoặc spread âm nên chỉ canonical daily được dùng làm target.
Global fair value	global_gold_usd_oz, usd_vnd, global_gold_vnd_per_luong	USD/oz, VND/USD, VND/lượng	Neo giá quốc tế sau khi đổi từ troy ounce sang lượng Việt Nam bằng 37.5 / 31.1034768.	LBMA historical thiếu licence nên một phần dùng GC=F/global close proxy và phải gắn cờ nguồn.
Premium SJC	premium, premium_pct, source_quality, is_proxy	VND/lượng và %	Phần giá nội địa cao hơn fair value toàn cầu. Premium cao vừa phản ánh khan hiếm vừa làm điểm mua đắt hơn.	Nếu nguồn reference/FX bị thiếu hoặc proxy nhiều, kết luận premium chỉ nên đọc theo hướng directional.
Macro / FX	usd_vnd_mid, vix, dxy_index, treasury_10y_pct, oil_wti_usd_barrel	Daily/as-of	Mô tả điều kiện USD, lãi suất, rủi ro và chi phí cơ hội nắm giữ vàng.	Phải lag/as-of theo ngày công bố; không dùng dữ liệu đóng cửa quốc tế sau thời điểm quyết định tại Việt Nam.
Event / News	event_type, severity, news_count, sentiment, availability_from	Daily event panel	Đo cường độ thông tin và chính sách có thể làm thay đổi premium, spread hoặc nhu cầu safe-haven.	Backfill headline không được dùng như realtime signal nếu chưa chứng minh availability_from <= feature_date.
Targets	target_date_1m/3m/5m, exit_date_1m/3m/5m, net_return_1m/3m/5m	Tháng lịch	Lợi suất thực thi: mua theo sell_price hôm nay và bán lại theo buy_price tại ngày exit khả dụng sau target	Không được diễn giải 21/63/105 dòng là 1/3/5 tháng khi frame có một dòng mỗi ngày lịch.

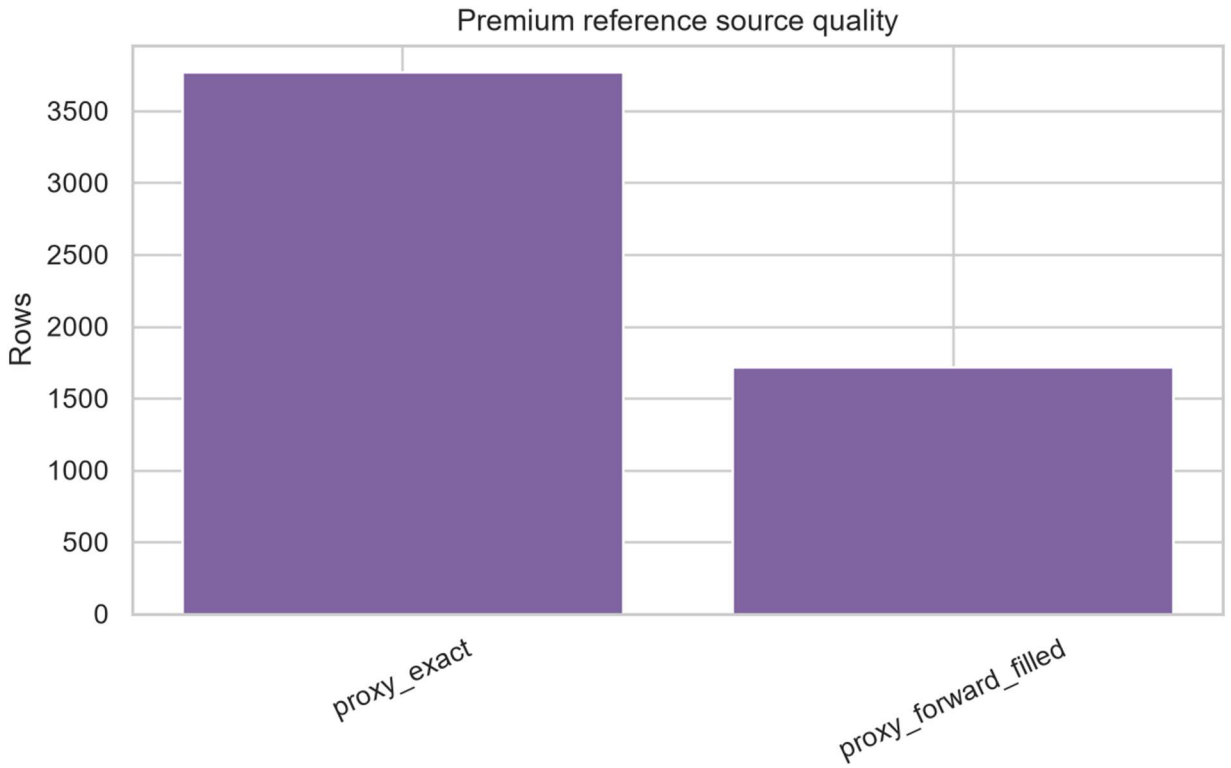
			month.	

Figure 0A. Field coverage trong model frame



Coverage chart cho biết biến nào thực sự có mặt trong frame huấn luyện. Đây là kiểm tra quan trọng trước khi bàn về model: một feature có ý nghĩa kinh tế tốt nhưng coverage thấp có thể làm model học không ổn định hoặc chỉ hoạt động trong một đoạn lịch sử hẹp.

Figure 0B. Source quality của premium/reference



Source-quality chart giúp tách câu hỏi 'có đủ dữ liệu để tính premium không' khỏi câu hỏi 'premium đó có phải benchmark chính thức không'. Báo cáo dùng proxy khi cần tính liên tục, nhưng luôn giữ cờ nguồn để sensitivity có thể tách high-quality-only khỏi proxy-inclusive.

3.2. Minh chứng dữ liệu crawl và artifact thu thập

Ngay trong phần collection, báo cáo trích một số dòng thật từ data lake để chứng minh nguồn dữ liệu đã được crawl/chuẩn hóa. Các bảng này không phải ví dụ minh họa tự tạo: chúng được đọc trực tiếp từ CSV trong data/lake khi build DOCX.

SBV source discovery sample: structure 137473 được phân loại central_fx, còn các structure liên quan vàng được giữ ở mức candidate nếu chưa có payload hợp lệ.

content_structure_id	classification	http_status	row_count_sample	field_names	title_samples
10100	gold_policy_candidate	403	0		
20117	gold_policy_candidate	403	0		
29633	gold_policy_candidate	403	0		
30000	gold_policy_candidate	403	0		
32827	gold_policy_candidate	403	0		
32829	gold_policy_candidate	403	0		

SBV gold-policy event sample: event chỉ được giữ khi có URL chính thức, ngày công bố, loại sự kiện và raw hash để audit.

event_date	published_at	event_type	severity	title	confidence	source_type
2026-07-16	2026-07-16	policy_auction	high	Giấy mời tham gia đấu thầu vàng	0.92	official_sbv_article
2026-07-16	2026-07-16	gold_sale_policy	medium	Quản lý hoạt động ngoại hối và hoạt động kinh doanh vàng	0.92	official_sbv_article
2026-07-16	2026-07-16	gold_sale_policy	medium	Quản lý hoạt động ngoại hối và hoạt động kinh doanh vàng	0.92	official_sbv_article

Retail deposit-rate sample: bảng này chứng minh lớp lãi suất hiện chỉ là forward monitoring, chưa phải lịch sử benchmark đầy đủ.

date	bank	tenor_months	currency	rate_pct_annual	published_at	available_from	history_status
2026-07-16	vietinbank	1	VND	2.1	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	2	VND	2.1	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	3	VND	2.4	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	4	VND	2.4	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	5	VND	2.4	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	6	VND	3.5	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	7	VND	3.5	2026-07-16	2026-07-16	forward_monitoring_only
2026-	vietinbank	8	VND	3.5	2026-07-16	2026-07-16	forward_monitoring_only

07-16							

LBMA append sample: dữ liệu public today.json được lưu riêng, còn lịch sử thiếu licence vẫn đọc bằng proxy có gắn cờ.

date	series_id	value	unit	source	available_from	fix_type
2026-07-16	LBMA_GOLD_AM_USD_OZ	4015.9	USD/oz	lbma_today_json	2026-07-16	AM
2026-07-16	LBMA_GOLD_PM_USD_OZ	4075.5	USD/oz	lbma_today_json	2026-07-16	PM
2026-07-16	LBMA_PALLADIUM_AM_USD_OZ	1269.15	USD/oz	lbma_today_json	2026-07-16	AM
2026-07-16	LBMA_PALLADIUM_PM_USD_OZ	1281.0	USD/oz	lbma_today_json	2026-07-16	PM
2026-07-16	LBMA_PLATINUM_AM_USD_OZ	1606.75	USD/oz	lbma_today_json	2026-07-16	AM
2026-07-16	LBMA_PLATINUM_PM_USD_OZ	1630.65	USD/oz	lbma_today_json	2026-07-16	PM

News availability sample: bảng này giải thích vì sao headline được tách thành research mode và strict realtime mode.

crawl_date	event_date	headline	published_at	fetched_at	availability_from	feature_mode_strict
2026-07-11	2026-07-09	Giá vàng hôm nay 10/7/2026: Giá vàng miếng, nhẫn trơn SJC 9999 và giá vàng thế giới thứ Sáu - baonghean.vn	2026-07-09	2026-07-11	2026-07-11	backfilled_not_realtime
2026-07-11	2026-07-10	Giá vàng hôm nay 11/7/2026 đà phục hồi bị chặn, kim loại quý đã 'chạm đáy', Trung Quốc tăng tốc gom hàng vì lý do gì?...	2026-07-10	2026-07-11	2026-07-11	strict_realtime_verified
2026-07-11	2026-07-11	Giá vàng hôm nay (trưa 11-7): Giá vàng nhẫn giảm - Báo Quân đội nhân dân	2026-07-11	2026-07-11	2026-07-11	strict_realtime_verified
2026-07-11	2026-07-11	Cập nhật giá vàng hôm nay 11/7/2026: Giá vàng 9999, nhẫn trơn SJC PNJ BTMC thứ Bảy - Báo Lâm Đồng	2026-07-11	2026-07-11	2026-07-11	strict_realtime_verified
2026-07-11	2026-07-10	Giá vàng hôm nay 11/7: Giá vàng trong nước đứng im, vàng thế giới giảm - VOV.VN	2026-07-10	2026-07-11	2026-07-11	strict_realtime_verified

3.3. Code crawl SBV đi kèm kết quả discovery

SBV discovery bằng Playwright và endpoint headless-delivery (scripts/pipeline/discover_sbv_sources.py:35-72)

Đoạn code này giải thích vì sao báo cáo chọn Playwright cho SBV: mở trang chính thức, tìm content-structure id và fetch endpoint qua browser context.

```
0035: SEED_URLS = [
0036:     "https://sbv.gov.vn/vi/",
0037:     "https://sbv.gov.vn/vi/bieu-do-ty-gia-trung-tam",
0038:     "https://sbv.gov.vn/vi/quan-ly-hoat-dong-ngoai-hoi-va-hoat-dong-kinh-doanh-vang",
0039: ]
0040:
0041: # Known verified structure: central USD/VND rate, not deposit rates.
0042: KNOWN_STRUCTURE_IDS = {"137473"}
0043:
0044:
0045: def content_fields(item: dict[str, Any]) -> dict[str, str]:
0046:     fields: dict[str, str] = {}
0047:     for field in item.get("contentFields", []) or []:
0048:         value = field.get("contentFieldValue") or {}
0049:         fields[str(field.get("name", ""))] = str(value.get("data", "") or "")
0050:     return fields
0051:
0052:
0053: def classify_structure(field_names: list[str], titles: list[str], seed_hits: list[str]) -> tuple[str,
str]:
0054:     haystack = " ".join(field_names + titles + seed_hits).lower()
0055:     if {"tygiaso", "tygiachu", "ngaybatdau"} & {f.lower() for f in field_names}:
0056:         return "central_fx", "verified: fields match SBV central USD/VND rate"
0057:     if any(k in haystack for k in ["lãi suất", "lai suat", "interest", "refinance", "tái cấp vốn", "liên
ngân hàng"]):
0058:         return "interest_rate_candidate", "candidate: text mentions SBV/market interest rates"
0059:     if any(k in haystack for k in ["đấu thầu vàng", "dau thau vang", "vàng miếng", "sjc", "kinh doanh
vàng"]):
0060:         return "gold_policy_candidate", "candidate: text mentions gold auction/SJC/gold policy"
0061:     if any(k in haystack for k in ["thông tư", "quyết định", "chỉ thị", "quy che", "quy chế"]):
0062:         return "policy_document_candidate", "candidate: text mentions official policy documents"
0063:     return "unknown", "not enough evidence to classify"
0064:
0065:
0066: def browser_fetch_structure(page: Any, structure_id: str, page_size: int) -> dict[str, Any]:
0067:     endpoint = f"/vi/o/headless-delivery/v1.0/content-structures/{structure_id}/structured-
contents?pageSize={page_size}&sort=datePublished:desc"
0068:     result = page.evaluate(
0069:         """
0070:         async ({endpoint}) => {
0071:           const response = await fetch(endpoint, {method: "GET"});
0072:           const text = await response.text();
```

Chuẩn hóa event chính sách vàng từ SBV (scripts/pipeline/collect_sbv_gold_policy_events.py:180-199)

Mỗi event đi vào bảng SBV policy phải có event_date/published_at, event_type, source_url, source_type, confidence và raw_hash.

```
0180:     event_type, severity = infer_event_type(hay)
0181:     published = parse_date_from_text(body) or parse_date_from_text(title) or ""
0182:     event_date = published
0183:     headline = normalize_text(candidate.get("anchor_text")) or title
0184:     if len(headline) < 10:
0185:         # Use the first body line that looks like a headline.
0186:         headline = normalize_text(body.split(".")[0])[:240] or title
0187:     confidence = 0.92 if published and status and 200 <= int(status) < 300 else 0.70
0188:     return {
0189:         "event_date": event_date,
0190:         "published_at": published,
0191:         "title": headline[:280],
0192:         "event_type": event_type,
0193:         "severity": severity,
0194:         "source_url": url,
0195:         "source_type": "official_sbv_article",
0196:         "confidence": confidence,
```

```
0197:         "raw_hash": hashlib.sha256(html.encode("utf-8", errors="replace")).hexdigest(),
0198:         "note": f"seed={candidate.get('seed_url', '')}; http_status={status}",
0199:     }
```

4. Data Quality và rủi ro dữ liệu

Data quality được đánh giá theo bốn nhóm: độ phủ theo thời gian, tính duy nhất ở grain ngày, tỷ lệ missing của biến quyết định, và tính đúng nghĩa của nguồn. Đây là điểm đặc biệt quan trọng với các nguồn SBV/NHNN: cùng một cấu trúc CMS có thể là tỷ giá trung tâm chứ không phải lãi suất tiền gửi, nên pipeline không được phép đổi nhãn ngữ nghĩa chỉ để có thêm feature.

Sau vòng cải thiện, premium coverage đã được xử lý bằng hierarchy nguồn có cờ proxy thay vì bỏ trống hàng loạt. Cách làm này không biến proxy thành benchmark chính thức; nó chỉ giúp mô hình có một đại lượng fair-value liên tục hơn, đồng thời vẫn giữ metadata để phân biệt ngày nào là nguồn chất lượng cao và ngày nào là proxy cần đọc thận trọng.

Check	Kết quả	Mức độ	Ý nghĩa
Target price validity	2 dòng bất thường	Cao	Phải bằng 0 hoặc được giải thích trước khi dùng làm label.
Model frame grain	5,485 dòng, 5,485 ngày unique	Cao	Một dòng/ngày cho training frame.
As-of guard: global_feature_date	PASS	Cao	Không dùng dữ liệu tương lai so với ngày quyết định.
As-of guard: gpr_feature_date	PASS	Cao	Không dùng dữ liệu tương lai so với ngày quyết định.
As-of guard: macro_feature_date	PASS	Cao	Không dùng dữ liệu tương lai so với ngày quyết định.
Premium missing	0.0%	Cao	Caveat lớn cho diễn giải premium và cơ hội entry.
Trading signal coverage	3,065 dòng	Trung bình	Bảng collapse đã xuất lại sau modeling run.
Runtime blocker	VECM cointegration screen did_not_pass; VECM forecast not promoted in v1 notebook.	Trung bình	Ghi vào báo cáo, không thay bằng kết quả giả.
Runtime blocker	Data diagnostics: {"raw_news_headline_rows": 3441, "raw_news_headline_unique_days": 702, "raw_news_headline_coverage": 0.12798541476754785, "raw_news_modeling_status": "included_ev	Trung bình	Ghi vào báo cáo, không thay bằng kết quả giả.

Premium coverage audit sample: kết quả cải thiện premium được đo theo năm và theo source quality, không chỉ ghi bằng lời.

segment	rows	premium_missing	premium_missing_rate	proxy_share	section
2011	179	0.0	0.0	1.0	by_year
2012	366	0.0	0.0	1.0	by_year
2013	365	0.0	0.0	1.0	by_year
2014	365	0.0	0.0	1.0	by_year
2015	365	0.0	0.0	1.0	by_year
2016	366	0.0	0.0	1.0	by_year
2017	365	0.0	0.0	1.0	by_year
2018	365	0.0	0.0	1.0	by_year

Premium coverage audit sau fallback có giới hạn (scripts/pipeline/improve_premium_coverage.py:257-287)

Đoạn code này cho thấy fallback được kiểm bằng missing rate, proxy_share và target <10%; nếu không đạt thì ghi blocker.

```
0257:
0258:     new_missing_rate = float(out["premium"].isna().mean())
0259:     by_year = out.assign(year=pd.to_datetime(out["date"]).dt.year).groupby("year", as_index=False).agg(
0260:         rows=("date", "size"),
0261:         premium_missing=("premium", lambda s: int(pd.to_numeric(s, errors="coerce").isna().sum())),
0262:         premium_missing_rate=("premium", lambda s: float(pd.to_numeric(s,
errors="coerce").isna().mean())),
0263:         proxy_share=("is_proxy", lambda s: float(pd.Series(s).astype(bool).mean())),
0264:     )
0265:     by_quality = out.groupby("source_quality", as_index=False).agg(rows=("date", "size"))
0266:     audit = pd.concat(
0267:         [
0268:             by_year.assign(section="by_year").rename(columns={"year": "segment"}),
0269:             by_quality.assign(section="by_source_quality").rename(columns={"source_quality":
"segment"}),
0270:         ],
0271:         ignore_index=True,
0272:         sort=False,
0273:     )
0274:     summary = {
0275:         "generated_at": datetime.now(timezone.utc).isoformat(),
0276:         "rows": int(len(out)),
0277:         "old_rows": int(old_rows),
0278:         "old_premium_missing_rate": old_missing_rate,
0279:         "new_premium_missing_rate": new_missing_rate,
0280:         "target_missing_rate": args.target_missing_rate,
0281:         "target_met": bool(new_missing_rate <= args.target_missing_rate),
0282:         "gold_tolerance_days": args.gold_tolerance_days,
0283:         "fx_tolerance_days": args.fx_tolerance_days,
0284:         "source_quality_counts": out["source_quality"].value_counts(dropna=False).to_dict(),
0285:         "blockers": [] if new_missing_rate <= args.target_missing_rate else [
0286:             "Premium missing remains above target after bounded as-of fallback; inspect source gaps by
year."
0287:         ],
```

5. Data Preparation

Quy trình chuẩn bị dữ liệu được thiết kế như một as-of pipeline. Tại mỗi ngày quyết định t, mô hình chỉ được thấy thông tin đã tồn tại trước hoặc tại thời điểm đó theo lịch công bố hợp lý. Với thị trường Mỹ, dữ liệu đóng cửa sau giờ giao dịch Việt Nam nên được lag tối thiểu một ngày. Với macro/news, trường available_from được dùng để mô phỏng ngày thông tin thực sự có thể được biết.

- Historical-valid target: chỉ dùng bản ghi có requested/business date đúng ngày và giá mua/bán hợp lệ.
- As-of join: global và GPR dùng cutoff t-1; macro dùng available_from <= date để tránh look-ahead bias.
- Feature engineering: tạo lag 1/5/10/21/63/105 ngày và rolling mean/std 5/21/63 ngày.
- Target: net_return_h = future_buy_price_h / current_sell_price - 1, đúng logic người mua vàng vật chất chịu spread.
- Horizon: 1/3/5 tháng lịch, lấy giá exit ở ngày khả dụng gần nhất sau target date.

Các đặc trưng rolling được tạo ở nhiều cửa sổ theo ngày vì mỗi horizon tháng có bản chất khác nhau. Horizon 1 tháng nhạy với momentum ngắn hạn và spread tức thời; horizon 3 tháng phản ánh pha điều chỉnh trung hạn; horizon 5 tháng chịu tác động mạnh hơn của chế độ vĩ mô, premium và sự kiện chính sách. Việc dùng cùng một bộ feature cho cả ba horizon giúp so sánh nhất quán, nhưng kết quả được đánh giá riêng từng horizon.

5.1. Code tạo target và chống leakage

Target lợi suất sau spread (src/gold_collectors/modeling/decision_support.py:386-430)

Đây là đoạn biến bài toán dự báo giá thành bài toán lợi suất thực thi: mua theo sell_price hiện tại và thoát theo future buy_price.

```
0386: def _add_targets(frame: pd.DataFrame, horizons: tuple[int, ...]) -> pd.DataFrame:
0387:     frame = frame.sort_values("date").reset_index(drop=True).copy()
0388:     dates = pd.to_datetime(frame["date"], errors="coerce").reset_index(drop=True)
0389:     buy = pd.to_numeric(frame["buy_price"], errors="coerce").reset_index(drop=True)
0390:     sell = pd.to_numeric(frame["sell_price"], errors="coerce").reset_index(drop=True)
0391:
0392:     for horizon in horizons:
0393:         target_dates = dates + pd.DateOffset(months=int(horizon))
0394:         exit_dates: list[pd.Timestamp | pd.NaT] = []
0395:         future_buy_values: list[float] = []
0396:         future_sell_values: list[float] = []
0397:         future_min_buy_values: list[float] = []
0398:         holding_days: list[float] = []
0399:
0400:         for idx, target_date in enumerate(target_dates):
0401:             if pd.isna(target_date):
0402:                 exit_dates.append(pd.NaT)
0403:                 future_buy_values.append(np.nan)
0404:                 future_sell_values.append(np.nan)
0405:                 future_min_buy_values.append(np.nan)
0406:                 holding_days.append(np.nan)
0407:                 continue
0408:
0409:             exit_pos = int(dates.searchsorted(target_date, side="left"))
0410:             if exit_pos >= len(frame):
0411:                 exit_dates.append(pd.NaT)
0412:                 future_buy_values.append(np.nan)
0413:                 future_sell_values.append(np.nan)
0414:                 future_min_buy_values.append(np.nan)
0415:                 holding_days.append(np.nan)
0416:                 continue
0417:
0418:             exit_dates.append(dates.iloc[exit_pos])
0419:             future_buy_values.append(float(buy.iloc[exit_pos]))
0420:             future_sell_values.append(float(sell.iloc[exit_pos]))
0421:             window = buy.iloc[idx + 1 : exit_pos + 1]
0422:             future_min_buy_values.append(float(window.min()) if not window.empty else np.nan)
0423:             holding_days.append(float((dates.iloc[exit_pos] - dates.iloc[idx]).days))
0424:
0425:         frame[f"target_date_{horizon}m"] = target_dates
0426:         frame[f"exit_date_{horizon}m"] = exit_dates
0427:         frame[f"holding_days_{horizon}m"] = holding_days
0428:         frame[f"future_buy_{horizon}m"] = future_buy_values
0429:         frame[f"future_sell_{horizon}m"] = future_sell_values
0430:         frame[f"net_return_{horizon}m"] = frame[f"future_buy_{horizon}m"] / frame["sell_price"] - 1
```

As-of join cho dữ liệu global (src/gold_collectors/modeling/decision_support.py:80-112)

Global market dùng cutoff t-1 trước khi merge_asof, giảm rủi ro dùng giá quốc tế đóng cửa sau giờ Việt Nam.

```
0080:     target = _safe_numeric(target, ["buy", "sell", "spread"])
0081:     target = target.dropna(subset=["date", "buy", "sell"])
0082:     target = target[(target["buy"] > 0) & (target["sell"] >= target["buy"])]
0083:     target = target.sort_values(["date", "timestamp_sort"]).groupby("date", as_index=False).tail(1)
0084:     target = target.rename(columns={"buy": "buy_price", "sell": "sell_price"})
0085:     target["mid_price"] = (target["buy_price"] + target["sell_price"]) / 2
0086:     target["spread_abs"] = target["sell_price"] - target["buy_price"]
0087:     target["spread_pct"] = target["spread_abs"] / target["sell_price"]
0088:     return target[["date", "buy_price", "sell_price", "mid_price", "spread_abs", "spread_pct"]]
0089:
0090:
0091: def _merge_global_asof(frame: pd.DataFrame, config: ModelingConfig) -> pd.DataFrame:
0092:     global_ref = _read_csv(_path(config, "pipeline_output_global_reference.csv"))
0093:     if global_ref.empty:
0094:         frame["global_feature_date"] = pd.NaT
0095:         return frame
0096:
```

```
0097:     global_ref["global_feature_date"] = _date_series(global_ref["date"])
0098:     numeric_cols = [
0099:         "dxy_index",
0100:         "gold_futures_close_usd_oz",
0101:         "lbma_price_usd_oz",
0102:         "oil_wti_usd_barrel",
0103:         "silver_futures_close_usd_oz",
0104:         "sp500_index",
0105:         "treasury_10y_pct",
0106:         "usd_vnd_market_rate",
0107:         "usd_vnd_mid",
0108:         "vix",
0109:     ]
0110:     global_ref = _safe_numeric(global_ref, numeric_cols)
0111:     keep = ["global_feature_date"] + [c for c in numeric_cols if c in global_ref.columns]
0112:     global_ref = global_ref[keep].sort_values("global_feature_date")
```

Deposit-rate as-of và opportunity cost (src/gold_collectors/modeling/decision_support.py:188-220)

Feature lãi suất chỉ được tạo khi có available_from, tenor và rate hợp lệ; nếu chưa có lịch sử thì không ép backfill.

```
0188:     keep_cols = ["macro_feature_date"] + [c for c in wide.columns if c != "macro_feature_date"][:25]
0189:     left = frame.sort_values("date").copy()
0190:     return pd.merge_asof(
0191:         left,
0192:         wide[keep_cols].sort_values("macro_feature_date"),
0193:         left_on="date",
0194:         right_on="macro_feature_date",
0195:         direction="backward",
0196:     )
0197:
0198:
0199: def _merge_deposit_asof(frame: pd.DataFrame, config: ModelingConfig) -> pd.DataFrame:
0200:     rates = _read_csv(_path(config, "normalized", "retail_deposit_rates.csv"))
0201:     if rates.empty:
0202:         return frame
0203:     rates = rates.copy()
0204:     rates["available_from"] = _date_series(rates.get("available_from", rates.get("date")))
0205:     rates["tenor_months"] = pd.to_numeric(rates.get("tenor_months"), errors="coerce")
0206:     rates["rate_pct_annual"] = pd.to_numeric(rates.get("rate_pct_annual"), errors="coerce")
0207:     rates = rates.dropna(subset=["available_from", "tenor_months", "rate_pct_annual"])
0208:     rates = rates[(rates["rate_pct_annual"] >= 0) & (rates["rate_pct_annual"] <= 20)]
0209:     if rates.empty:
0210:         return frame
0211:
0212:     horizon_to_tenor = {1: 1, 3: 3, 5: 5}
0213:     out = frame.sort_values("date").copy()
0214:     for horizon, tenor in horizon_to_tenor.items():
0215:         sub = (
0216:             rates[rates["tenor_months"].astype(int).eq(tenor)]
0217:             .groupby("available_from", as_index=False)["rate_pct_annual"]
0218:             .mean()
0219:             .sort_values("available_from")
0220:         )
```

Pipeline build model frame (src/gold_collectors/modeling/decision_support.py:465-490)

Thứ tự build frame thể hiện toàn bộ pipeline: target SJC, premium, global, GPR, macro, deposit, event, news, target và lagged features.

```
0465:     ]
0466:     base_cols += [c for c in frame.columns if c.startswith(("event_count", "severity_", "channel_",
"days_until_next_"))]
0467:     base_cols += [c for c in frame.columns if c.startswith("raw_news_")]
0468:     base_cols += [
0469:         c
0470:         for c in frame.columns
0471:         if c
0472:         in {
```

```
0473:             "imports_all_cif_m_usd",
0474:             "imports_direct_m_usd",
0475:             "vnindex_eop",
0476:             "cpi_monthly_index",
0477:             "ip_total_index",
0478:             "unemployment_rate_pct",
0479:         }
0480:     ]
0481:     base_cols = [c for c in dict.fromkeys(base_cols) if c in frame.columns]
0482:
0483:     feature_cols: list[str] = []
0484:     new_features: dict[str, pd.Series] = {}
0485:     for col in base_cols:
0486:         frame[col] = pd.to_numeric(frame[col], errors="coerce")
0487:         if col.startswith(("event_", "severity_", "channel_", "days_until_next_")):
0488:             feature_cols.append(col)
0489:             continue
0490:         for lag in config.lags:
```

Model frame sample ở cuối snapshot: dữ liệu đã được ghép cùng grain ngày, kèm target tương lai cho từng horizon.

date	buy_price	sell_price	spread_pct	premium	premium_pct	is_proxy	net_return_1m	net_return_3m	net_return_5m
2026-07-07	147000000.0	150000000.0	0.02	19563032.28	0.14884	1.0			
2026-07-08	145500000.0	148500000.0	0.020202020202020202	20971148.83	0.162531	1.0			
2026-07-09	146000000.0	149000000.0	0.020134228187919194	17554032.41	0.134056	1.0			
2026-07-10	146900000.0	149900000.0	0.020013342228521521	18802740.99	0.144417	1.0			
2026-07-11	146900000.0	149900000.0	0.020013342228521521	19702740.99	0.15133	1.0			

5.2. Methodology notebook: công thức, artifact và validation gate

Khối này trình bày theo kiểu notebook: mỗi bước có input artifact, phép biến đổi, output và cổng kiểm tra. Mục tiêu là để người đọc không chỉ thấy kết luận, mà còn thấy con đường dữ liệu đi từ raw crawl tới forecast và quyết định mua.

Đại lượng	Công thức / rule	Ý nghĩa	Validation gate
Global gold VND/lượng	gold_usd_oz * usd_vnd * 37.5 / 31.1034768	Đổi USD/troy oz sang VND/lượng Việt Nam. Đây là lỗi nghiêm trọng đã sửa so với logic 1.205 gram.	stored/correct median phải quanh 1.0, không quanh 1.205.
Premium	sell_consensus - global_gold_vnd_per_luong; premium_pct = premium / global_gold_vnd_per_luong	Đo phần chênh giá SJC so với fair value toàn cầu quy đổi.	premium_pct không âm hàng loạt bất hợp lý sau khi sửa đơn vị; missing được audit theo năm/source.
Spread	sell_price - buy_price; spread_pct = spread / mid_price	Chi phí round-trip tối thiểu của người mua vàng vật chất.	Canonical target loại buy<=0, sell<=0 và sell<buy.

Target month	target_date_h = date + h calendar months; exit_date_h = first available price date >= target_date_h	Đúng nghĩa 1/3/5 tháng lịch thay vì số dòng trong daily panel.	exit_date >= target_date; holding_days được phân phối quanh tháng lịch tương ứng.
Net return	future_buy_price_h / current_sell_price - 1	Lợi suất thực thi sau spread: mua theo giá bán, thoát theo giá mua.	Target non-null được đếm riêng từng horizon và không dùng hàng chưa có exit.
Decision rule	buy nếu expected_return > 0, P(return>0) đủ cao và q10 downside không vượt sàn rủi ro	Kết hợp kỳ vọng, xác suất thắng và kịch bản xấu q10.	Signal được backtest theo validation/test và ghi paper-trading nếu sau snapshot.

Bảng công thức trên là phần xương sống của báo cáo. Điểm quan trọng nhất là target và premium đều mô phỏng hành vi mua bán có thể thực thi: premium dựa trên VND/lượng đúng đơn vị, còn return dựa trên giá mua lại trong tương lai chứ không phải giá mid lý tưởng.

Công thức quy đổi premium đúng đơn vị (scripts/pipeline/improve_premium_coverage.py:200-209)

Đoạn code này là bằng chứng kỹ thuật cho công thức gold_usd_oz * USD/VND * 37.5 / 31.1034768.

```
0200:     fx_ref = load_fx_reference()
0201:     out = asof_attach(domestic, gold_ref, "gold", args.gold_tolerance_days)
0202:     out = asof_attach(out, fx_ref, "fx", args.fx_tolerance_days)
0203:
0204:     out["global_gold_vnd_per_luong"] = out["global_gold_usd_oz"] * out["usd_vnd"] * OZ_PER_LUONG
0205:     missing = out["global_gold_vnd_per_luong"].isna()
0206:     out.loc[missing, "premium"] = np.nan
0207:     out.loc[~missing, "premium"] = out.loc[~missing, "sell_consensus"] - out.loc[~missing,
"global_gold_vnd_per_luong"]
0208:     out["premium_pct"] = out["premium"] / out["global_gold_vnd_per_luong"]
0209:     out["source_quality"] = out.apply(quality_label, axis=1)
```

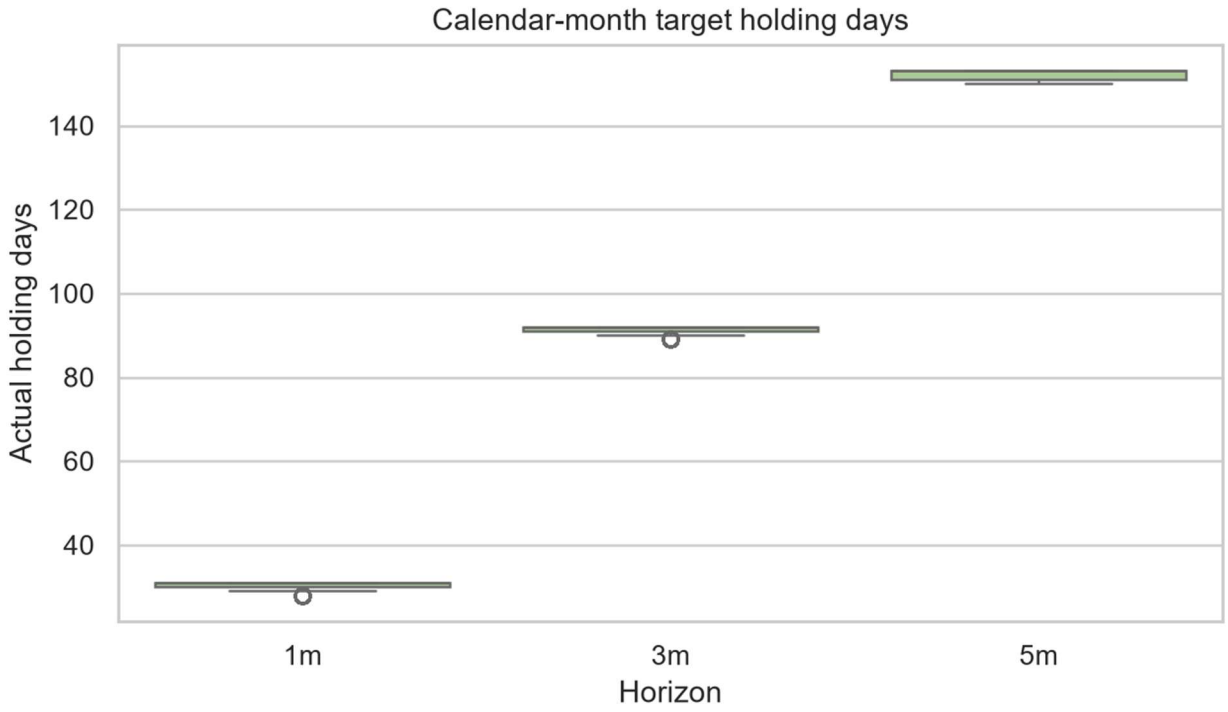
Target theo tháng lịch và ngày exit khả dụng (src/gold_collectors/modeling/decision_support.py:386-430)

Target không còn dùng shift theo số dòng; mỗi horizon lấy target_date theo tháng lịch rồi tìm ngày giá khả dụng gần nhất sau đó.

```
0386: def _add_targets(frame: pd.DataFrame, horizons: tuple[int, ...]) -> pd.DataFrame:
0387:     frame = frame.sort_values("date").reset_index(drop=True).copy()
0388:     dates = pd.to_datetime(frame["date"], errors="coerce").reset_index(drop=True)
0389:     buy = pd.to_numeric(frame["buy_price"], errors="coerce").reset_index(drop=True)
0390:     sell = pd.to_numeric(frame["sell_price"], errors="coerce").reset_index(drop=True)
0391:
0392:     for horizon in horizons:
0393:         target_dates = dates + pd.DateOffset(months=int(horizon))
0394:         exit_dates: list[pd.Timestamp | pd.NaT] = []
0395:         future_buy_values: list[float] = []
0396:         future_sell_values: list[float] = []
0397:         future_min_buy_values: list[float] = []
0398:         holding_days: list[float] = []
0399:
0400:         for idx, target_date in enumerate(target_dates):
0401:             if pd.isna(target_date):
0402:                 exit_dates.append(pd.NaT)
0403:                 future_buy_values.append(np.nan)
0404:                 future_sell_values.append(np.nan)
0405:                 future_min_buy_values.append(np.nan)
0406:                 holding_days.append(np.nan)
0407:                 continue
0408:
0409:             exit_pos = int(dates.searchsorted(target_date, side="left"))
0410:             if exit_pos >= len(frame):
0411:                 exit_dates.append(pd.NaT)
0412:                 future_buy_values.append(np.nan)
0413:                 future_sell_values.append(np.nan)
```

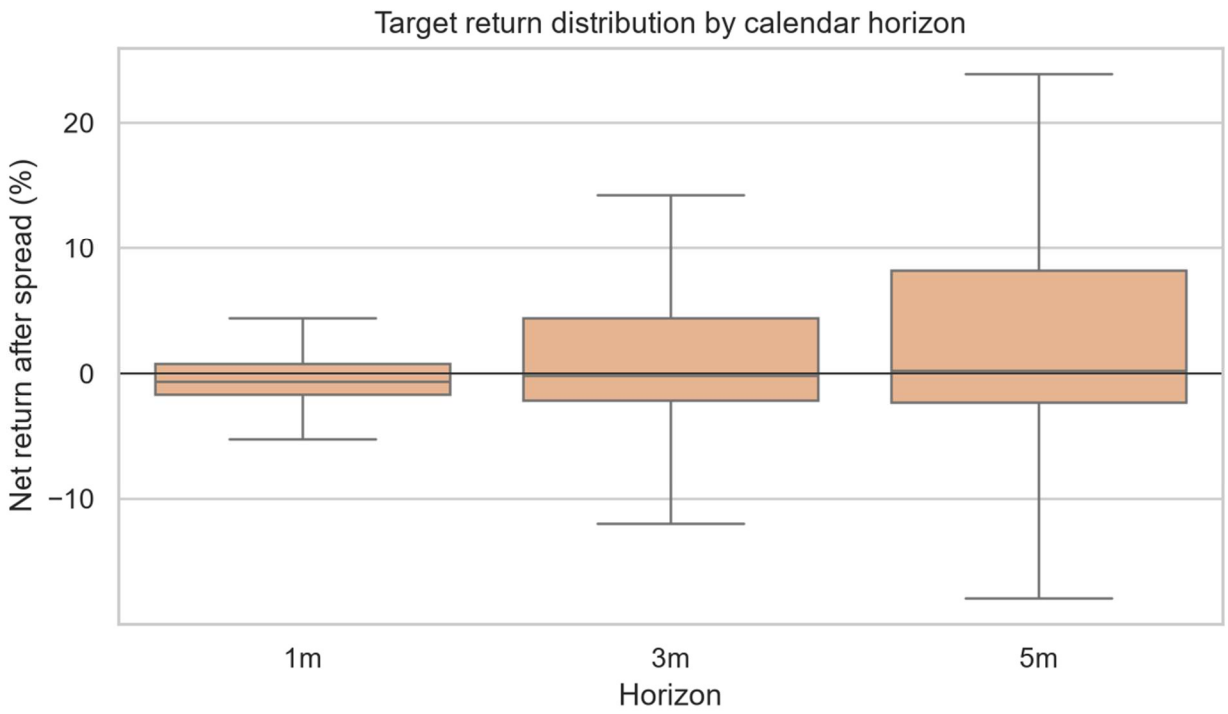
```
0414:                 future_min_buy_values.append(np.nan)
0415:                 holding_days.append(np.nan)
0416:                 continue
0417:
0418:             exit_dates.append(dates.iloc[exit_pos])
0419:             future_buy_values.append(float(buy.iloc[exit_pos]))
0420:             future_sell_values.append(float(sell.iloc[exit_pos]))
0421:             window = buy.iloc[idx + 1 : exit_pos + 1]
0422:             future_min_buy_values.append(float(window.min()) if not window.empty else np.nan)
0423:             holding_days.append(float((dates.iloc[exit_pos] - dates.iloc[idx]).days))
0424:
0425:         frame[f"target_date_{horizon}m"] = target_dates
0426:         frame[f"exit_date_{horizon}m"] = exit_dates
0427:         frame[f"holding_days_{horizon}m"] = holding_days
0428:         frame[f"future_buy_{horizon}m"] = future_buy_values
0429:         frame[f"future_sell_{horizon}m"] = future_sell_values
0430:         frame[f"net_return_{horizon}m"] = frame[f"future_buy_{horizon}m"] / frame["sell_price"] - 1
```

Figure 0C. Holding-day distribution của target tháng lịch



Holding-day distribution là kiểm tra trực quan cho lỗi horizon trước đây. Nếu 1/3/5 tháng được tạo đúng, số ngày nắm giữ phải xoay quanh tháng lịch tương ứng và có dao động do cuối tuần/ngày thiếu giá, không cố định cứng 21/63/105 dòng.

Figure 0D. Phân phối target return sau spread



Target distribution cho thấy bài toán không cân xứng: cùng một frame nhưng horizon dài có phân phối lợi suất khác horizon ngắn. Vì vậy report không dùng một model chung cho mọi horizon, mà train/evaluate riêng 1 tháng, 3 tháng và 5 tháng.

Bước	Input artifact	Transformation	Output artifact	Validation gate
Canonical domestic	raw/crawl domestic gold tables	Chuẩn hóa unit, chọn SJC daily target, loại giá 0 và spread âm	gold_quotes_sjc_historical.csv, gold_domestic_daily_panel.csv	unique by date trong canonical; no buy<=0, sell<=0, sell<buy
Reference/premium	global_reference_daily.csv, FX, LBMA/GC=F proxy	Quy đổi USD/oz sang VND/lượng, tính premium và proxy flag	pipeline_output_premium_enriched.csv	conversion audit pass; premium missing và proxy share được ghi
As-of feature frame	domestic, premium, macro, GPR, events, news	merge_asof/availability guard, tạo lag và rolling windows	model_frame_daily.csv	feature availability <= feature_date; unique by date
Calendar targets	model_frame_daily.csv + canonical prices	Tạo target_date/exit_date/holding_days/net_return cho 1/3/5 tháng	model_frame_daily.csv	exit_date >= target_date; target non-null count khớp summary
Modeling	model_frame_daily.csv, feature_columns.json	Walk-forward train/evaluate baseline, linear, SARIMAX, RF, boosting, quantile	model_results.csv, decision_signals.csv	Không random split;

				optional model chỉ công bố nếu train thật
Decision monitoring	snapshot_forecasts.csv, latest canonical prices	Ghi ledger sau snapshot; open nếu chưa đáo hạn	paper_trading_ledger.csv	Không tính realized return cho open trades

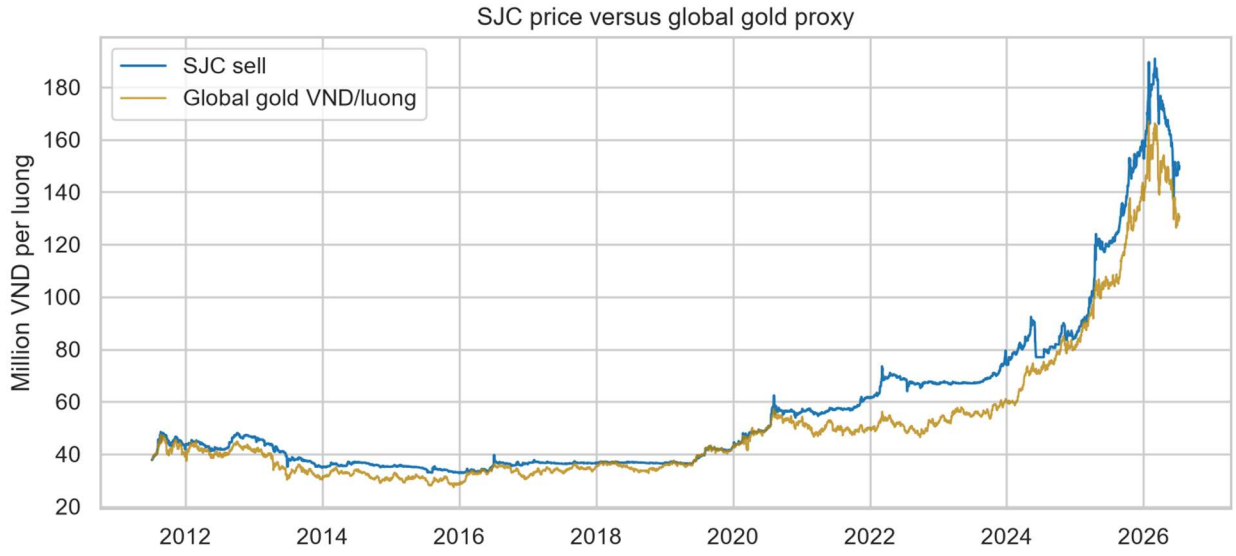
6. EDA: Giá, premium, thanh khoản và sự kiện

EDA tập trung vào những câu hỏi có ảnh hưởng trực tiếp đến quyết định mua: giá SJC có đang ở vùng biến động bất thường không, premium đang đắt hay rẻ so với lịch sử, spread có làm chi phí giao dịch quá cao không, và các sự kiện chính sách/news có trùng với thay đổi regime hay không. Các bảng phía dưới vì vậy được đọc như bằng chứng định hướng mô hình, không phải chỉ là thống kê mô tả.

Diễn biến 10 năm gần nhất của giá bán SJC trong model frame:

Năm	Start	End	Min	Max	Return	N obs	Avg spread
2017	36.10 triệu	36.44 triệu	36.10 triệu	37.75 triệu	0.9%	365	0.67%
2018	36.67 triệu	36.57 triệu	36.38 triệu	37.17 triệu	-0.3%	365	0.48%
2019	36.57 triệu	42.75 triệu	36.24 triệu	42.90 triệu	16.9%	365	0.60%
2020	42.75 triệu	56.10 triệu	42.64 triệu	62.40 triệu	31.2%	366	1.24%
2021	56.10 triệu	61.65 triệu	54.50 triệu	61.80 triệu	9.9%	365	1.02%
2022	61.65 triệu	66.70 triệu	61.45 triệu	73.50 triệu	8.2%	365	1.29%
2023	66.70 triệu	74.00 triệu	66.55 triệu	79.50 triệu	10.9%	365	1.10%
2024	74.00 triệu	84.20 triệu	74.00 triệu	92.40 triệu	13.8%	366	2.70%
2025	84.20 triệu	152.80 triệu	84.20 triệu	159.70 triệu	81.5%	365	1.75%
2026	152.80 triệu	149.90 triệu	138.30 triệu	190.90 triệu	-1.9%	192	1.75%

Figure 1. SJC price versus global gold proxy



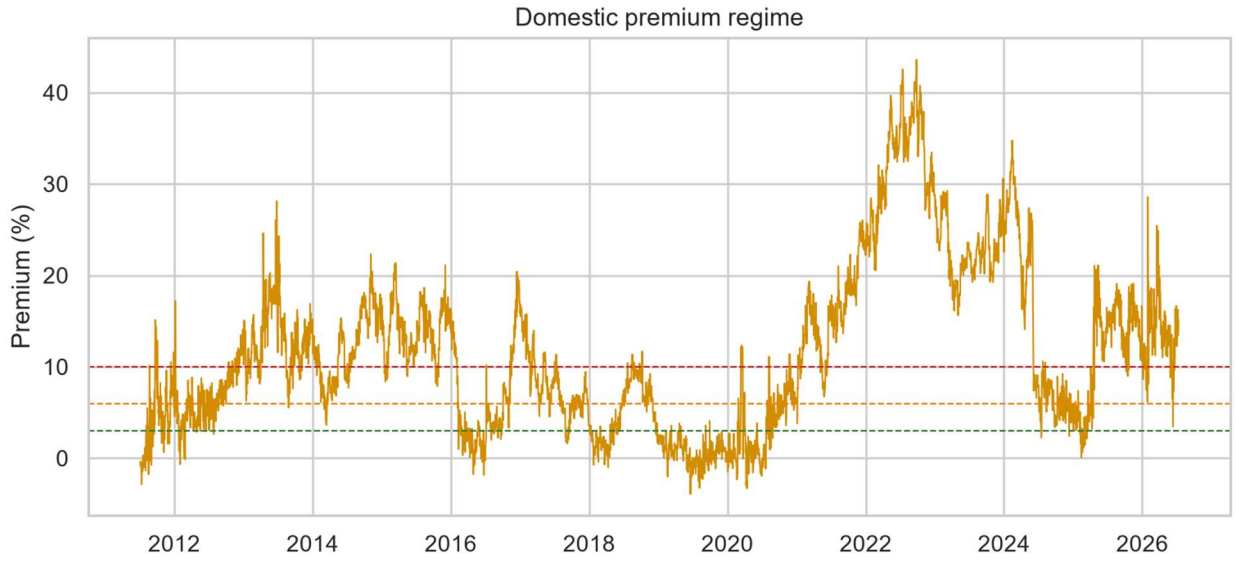
Bảng theo năm cho thấy vàng SJC không tăng đều tuyến tính; thay vào đó có các pha tăng mạnh, đi ngang và điều chỉnh. Vì thế mô hình không được đánh giá bằng một giai đoạn holdout ngắn duy nhất, mà cần walk-forward để

kiểm tra nhiều chế độ thị trường. Nếu một mô hình chỉ tốt trong pha tăng nhưng sai trong pha premium co lại, mô hình đó không đủ tin cậy cho quyết định mua.

Premium decomposition cho thấy premium là biến quyết định chất lượng điểm mua:

Metric	Premium VND/lượng	Premium %
Số ngày có premium	5,485	
Premium trung bình	6.31 triệu	11.64%
Premium median	4.01 triệu	10.21%
P10	0.47 triệu	1.20%
P90	16.76 triệu	24.22%

Figure 2. Premium regime over time



Premium được hiểu là phần giá nội địa cao hơn hoặc thấp hơn fair value toàn cầu quy đổi. Premium dương cao có thể phản ánh khan hiếm và nhu cầu tích trữ mạnh, nhưng cũng làm điểm mua trở nên đắt nếu chính sách cung vàng sau đó kéo premium xuống. Do đó premium vừa là biến giải thích xu hướng ngắn hạn, vừa là nguồn rủi ro mean-reversion.

Premium regime	Tỷ trọng quan sát
Thấp (<3%)	17.4%
Bình thường (3-6%)	13.1%
Cao (6-10%)	19.0%
Crisis (>10%)	50.5%

Global variable	Non-null	Coverage	Min	Max
gold_futures_close_usd_oz	3,774	86.6%	1,050.800	5,318.400
lbma_price_usd_oz	4,153	95.3%	1,050.800	5,318.400
usd_vnd_mid	3,969	91.1%	21.000	26,425.000
vix	3,969	91.1%	9.140	82.690
dxy_index	3,969	91.1%	73.470	114.110
treasury_10y_pct	3,969	91.1%	0.520	4.980
oil_wti_usd_barrel	3,969	91.1%	-37.630	123.700

Coverage của biến toàn cầu quyết định độ tin cậy của premium. Khi LBMA lịch sử không có licence đầy đủ, GC=F/global close proxy được dùng với cờ nguồn rõ ràng. Báo cáo không coi proxy là benchmark chính thức, nhưng sử dụng nó để giữ cấu trúc fair value liên tục cho mô hình và ghi caveat trong phần cải thiện dữ liệu.

Các loại sự kiện phổ biến trong event regime panel:

Event type	Count
wedding_season	1,530
tet_proximity	240
geopolitical_crisis	21
than_tai	16
eurozone_crisis	12
financial_stress	8
policy_import	6
financial_crisis	5
banking_stress	4
policy_inspection	4
policy_rate_increase	2
policy_auction	1

7. Modeling và Evaluation

Thiết kế evaluation dùng expanding-window walk-forward: train trên quá khứ, đánh giá trên validation/test theo thời gian. Đây là bắt buộc vì random split sẽ làm rò rỉ chế độ thị trường tương lai vào quá khứ.

Mỗi horizon được huấn luyện và đánh giá riêng vì phân phối lợi suất 1, 3 và 5 tháng không giống nhau. Horizon ngắn thường bị chi phối bởi nhiễu, spread và timing; horizon dài hơn có nhiều thời gian để xu hướng vĩ mô hoặc premium regime phát huy tác động, nhưng cũng chịu rủi ro chính sách lớn hơn. Vì vậy, việc một mô hình tốt ở 5 tháng không tự động hợp lệ cho 1 tháng.

Bộ mô hình được sắp theo mức phức tạp tăng dần. Baseline zero/mean/median cho biết mức sàn cần vượt qua; Ridge/ElasticNet kiểm tra quan hệ tuyến tính có regularization; SARIMAX kiểm tra cấu trúc chuỗi thời gian có exogenous; Random Forest và boosting kiểm tra phi tuyến; quantile models ước lượng downside. LightGBM, XGBoost và CatBoost chỉ được đưa vào leaderboard khi dependency import được và mô hình train thật.

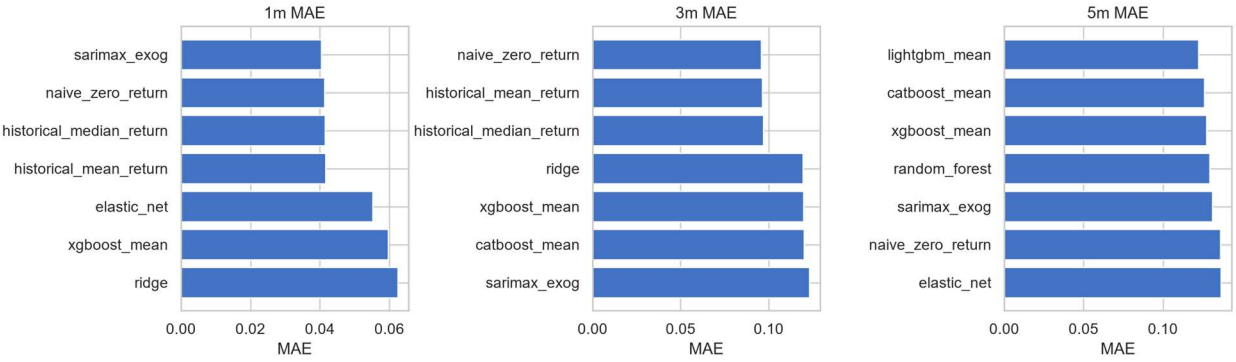
Các chỉ số thống kê như MAE/RMSE cho biết sai số trung bình, nhưng quyết định đầu tư cần thêm directional accuracy và signal performance. Một mô hình có MAE thấp nhưng thường dự báo sai hướng ở các ngày phát tín hiệu mua sẽ không hữu ích. Ngược lại, một mô hình không tối ưu MAE tuyệt đối nhưng phát tín hiệu ít, có chọn lọc và có q10 không quá xấu có thể tốt hơn về mặt quyết định.

Model family	Vai trò trong phân tích	Điều kiện đưa vào leaderboard	Caveat chính
Zero/mean/median baseline	Mốc sàn để biết model phức tạp có thật sự tạo thêm giá trị.	Luôn chạy được trên target non-null.	Không học regime; chỉ hữu ích làm benchmark.
Ridge / ElasticNet	Kiểm tra quan hệ tuyến tính có regularization giữa premium, FX, macro và target.	Train trên walk-forward folds với feature imputation.	Không bắt tốt tương tác phi tuyến hoặc threshold chính sách.
SARIMAX + exog / VECM-style checks	Đưa cấu trúc chuỗi thời gian và biến ngoại sinh vào baseline thống kê.	Chỉ công bố fold chạy ổn định; lỗi convergence được ghi blocker.	Nhạy với non-stationarity và regime break.
Random Forest / Gradient Boosting	Bắt phi tuyến giữa premium, spread, events, FX và rủi ro toàn cầu.	Phải train thật trên frame as-of; không dùng import thành công như kết quả.	Dễ overfit nếu signal quá dày hoặc feature coverage lệch theo thời gian.
LightGBM / XGBoost / CatBoost	Ứng viên tabular boosting mạnh cho feature nhiều cột và tương tác phức tạp.	Chỉ xuất hiện khi dependency cài được và model chạy xong trong pipeline.	Cần so với baseline bằng walk-forward, không chỉ nhìn one-shot score.
DeepAR / TFT	Ứng viên probabilistic multi-horizon sau khi strict-as-of	Chưa promote nếu runner/dependency/deployment	Không thay bằng kết quả giả; chỉ ghi hướng nâng cấp.

	frame ổn định.	chưa đủ chắc.	

Horizon	Best MAE model	MAE	RMSE	DA	Best DA model	Best DA
1 tháng	sarimax_exog	0.0403	0.0540	48.1%	historical_mean_return	53.7%
3 tháng	naive_zero_return	0.0957	0.1098	0.5%	historical_mean_return	60.5%
5 tháng	lightgbm_mean	0.1224	0.1473	47.7%	historical_mean_return	61.9%

Figure 3. Model MAE leaderboard by horizon



Bảng leaderboard được đọc theo hai lớp. Lớp thứ nhất là khả năng dự báo mức lợi suất, thể hiện qua MAE và RMSE. Lớp thứ hai là khả năng dự báo hướng, thể hiện qua directional accuracy. Nếu hai lớp này mâu thuẫn, báo cáo không chọn mô hình chỉ vì một chỉ số; quyết định cuối cùng còn phải đi qua rule xác suất dương và downside q10.

Optional model training cho LightGBM/XGBoost/CatBoost (src/gold_collectors/modeling/decision_support.py:746-792)

Đoạn code này nằm ngay cạnh leaderboard để chứng minh các mô hình boosting chỉ xuất hiện khi import được và train thật, không phải ghi giả trong báo cáo.

```
0746: def train_ml_models(
0747:     frame: pd.DataFrame,
0748:     feature_cols: list[str],
0749:     config: ModelingConfig | None = None,
0750: ) -> tuple[pd.DataFrame, pd.DataFrame, list[str]]:
0751:     config = config or ModelingConfig()
0752:     blockers: list[str] = []
0753:     try:
0754:         from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
0755:         from sklearn.linear_model import ElasticNet, Ridge
0756:         from sklearn.pipeline import make_pipeline
0757:         from sklearn.preprocessing import StandardScaler
0758:     except Exception as exc:  # pragma: no cover - depends on local env
0759:         return pd.DataFrame(), pd.DataFrame(), [f"scikit-learn unavailable: {exc}"]
0760:
0761:     feature_cols = _bounded_feature_subset(feature_cols)
0762:     optional_models: dict[str, Any] = {}
0763:     try:
0764:         from lightgbm import LGBMRegressor  # type: ignore
0765:
0766:         optional_models["lightgbm_mean"] = LGBMRegressor(n_estimators=300, learning_rate=0.03,
random_state=config.random_state, verbose=-1)
0767:         optional_models["lightgbm_q10"] = LGBMRegressor(
0768:             objective="quantile", alpha=0.10, n_estimators=300, learning_rate=0.03,
random_state=config.random_state, verbose=-1
0769:         )
0770:     except Exception as exc:
0771:         blockers.append(f"LightGBM unavailable; using sklearn quantile fallback: {exc}")
0772:     try:
0773:         from xgboost import XGBRegressor  # type: ignore
0774:
```

```
0775:         optional_models["xgboost_mean"] = XGBRegressor(
0776:             n_estimators=250, max_depth=3, learning_rate=0.03, subsample=0.9, colsample_bytree=0.9,
random_state=config.random_state
0777:         )
0778:     except Exception as exc:
0779:         blockers.append(f"XGBoost unavailable: {exc}")
0780:     try:
0781:         from catboost import CatBoostRegressor  # type: ignore
0782:
0783:         optional_models["catboost_mean"] = CatBoostRegressor(
0784:             iterations=300,
0785:             depth=4,
0786:             learning_rate=0.03,
0787:             loss_function="RMSE",
0788:             random_seed=config.random_state,
0789:             allow_writing_files=False,
0790:             verbose=False,
0791:         )
0792:         optional_models["catboost_q10"] = CatBoostRegressor(
```

Horizon	Best quantile model	Pinball loss
1 tháng	sklearn_gbr_q05	0.01119
3 tháng	sklearn_gbr_q05	0.02620
5 tháng	sklearn_gbr_q05	0.03430

Pinball loss dùng cho mô hình phân vị q10. q10 được hiểu là kịch bản xấu nhưng không cực đoan: nếu q10 quá âm, nhà đầu tư phải chấp nhận rằng trong khoảng 10% trường hợp xấu, khoản lỗ có thể vượt ngưỡng chịu đựng. Đây là lớp kiểm soát rủi ro quan trọng vì thị trường vàng Việt Nam có thể bị tác động bởi premium và chính sách bất ngờ.

Decision-rule backtest với rule P(return>0) >= 0.50 và q10 >= -10%:

Horizon	Phase	Signals	Signal rate	Avg buy-day ret	Avg strategy ret
1 tháng	test	32/365	8.8%	1.34%	0.12%
1 tháng	validation	0/365	0.0%	n/a	0.00%
3 tháng	test	328/466	70.4%	9.87%	6.95%
3 tháng	validation	306/731	41.9%	1.46%	0.61%
5 tháng	test	407/407	100.0%	19.95%	19.95%
5 tháng	validation	366/731	50.1%	7.05%	3.53%

Figure 4. Decision signal frequency

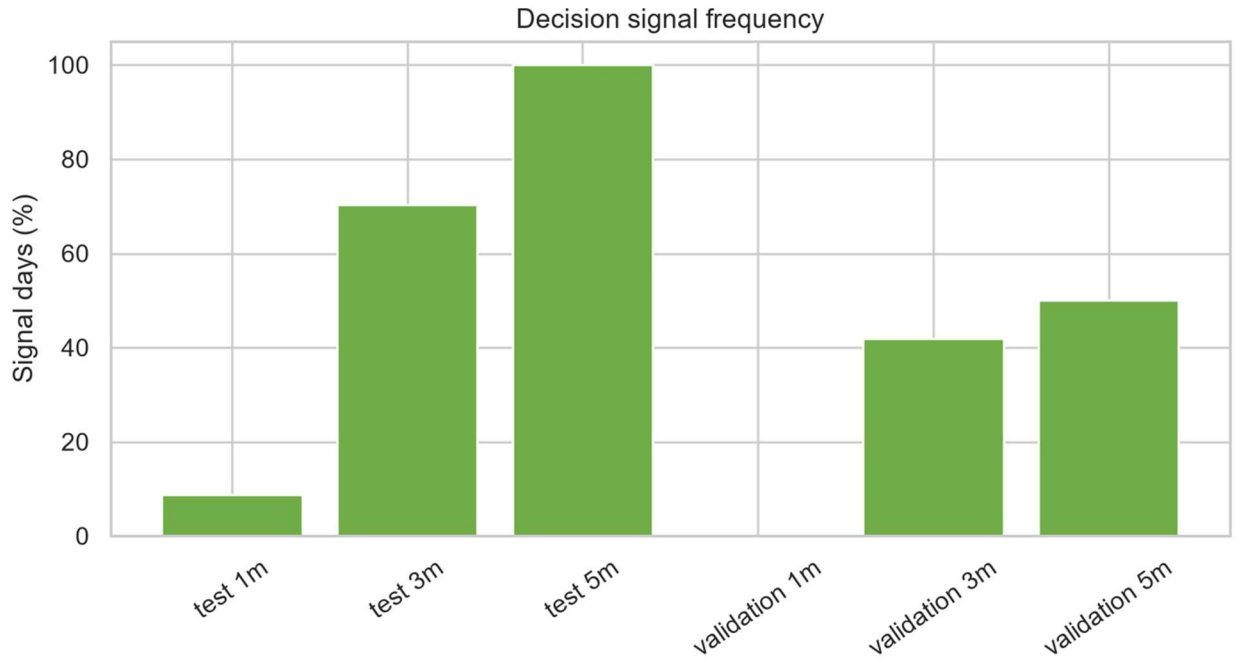
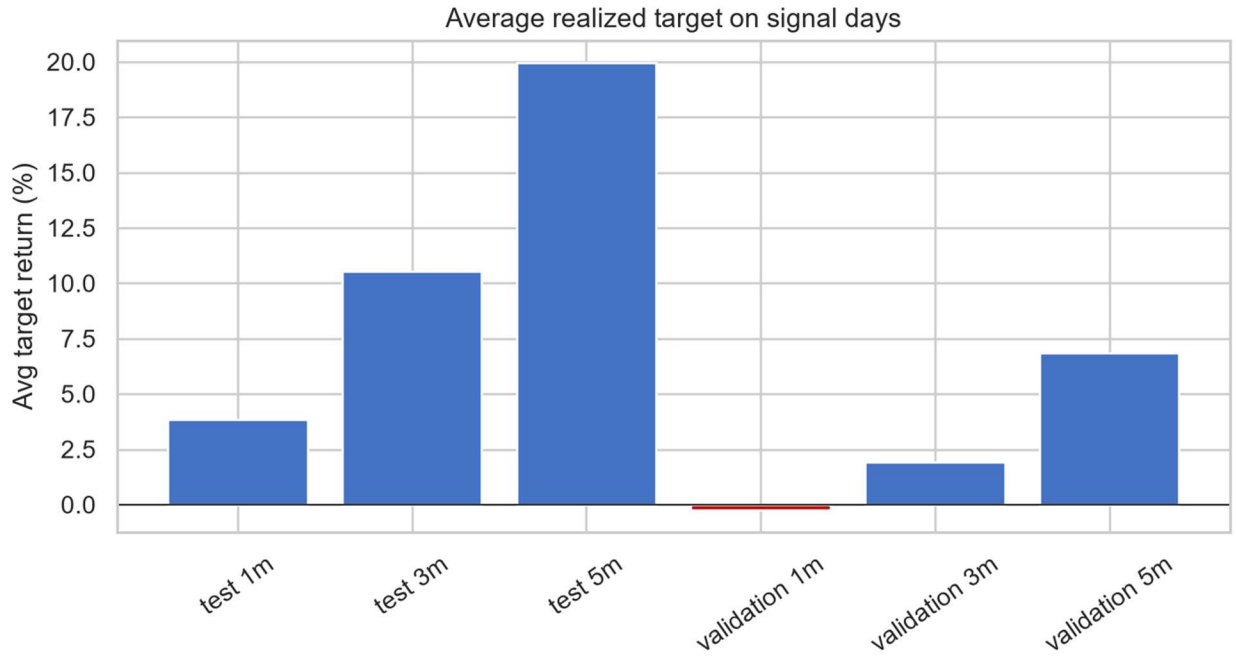


Figure 5. Average target return on signal days



Decision-rule backtest trả lời câu hỏi khác với leaderboard: nếu chỉ mua vào những ngày mô hình bật tín hiệu, kết quả trung bình của các ngày đó ra sao. Tỷ lệ phát tín hiệu quá cao có thể nghĩa là rule thiếu chọn lọc; tỷ lệ quá thấp có thể nghĩa là mô hình không đủ cơ hội thực thi. Vì vậy báo cáo đọc đồng thời signal rate, average buy-day return và consistency giữa validation/test.

Decision rule từ xác suất và q10 downside (src/gold_collectors/modeling/decision_support.py:960-990)

Tín hiệu mua không chỉ dựa vào expected return; rule yêu cầu P(return>0) vượt ngưỡng và q10 không thấp hơn sàn rủi ro.

```
0960:         selected = mean_preds[(mean_preds["horizon_months"].eq(horizon)) &
```

```
(mean_preds["model"].eq(model_name))].copy()
0961:         if selected.empty:
0962:             continue
0963:         residual_std = float((selected["actual_net_return"] -
selected["predicted_net_return"]).std(skipna=True))
0964:         residual_std = residual_std if np.isfinite(residual_std) and residual_std > 0 else 0.05
0965:         selected["prob_return_positive"] = 1.0 - selected["predicted_net_return"].apply(lambda x: 0.5 *
(1.0 + math.erf((0.0 - x) / (residual_std * math.sqrt(2)))))
0966:
0967:         q10 = predictions[
0968:             predictions["horizon_months"].eq(horizon)
0969:             & predictions["prediction_type"].eq("q10")
0970:             & predictions["model"].astype(str).str.contains("q10")
0971:         ].copy()
0972:         if not q10.empty:
0973:             q10 = q10.sort_values(["date", "model"]).drop_duplicates(["date", "horizon_months"],
keep="first")
0974:             selected = selected.merge(
0975:                 q10[["date", "horizon_months",
"predicted_net_return"]].rename(columns={"predicted_net_return": "q10_predicted_net_return"}),
0976:                 on=["date", "horizon_months"],
0977:                 how="left",
0978:             )
0979:         else:
0980:             selected["q10_predicted_net_return"] = selected["predicted_net_return"] - 1.2816 *
residual_std
0981:
0982:         selected["selected_model"] = model_name
0983:         for prob_threshold in (0.55, config.decision_prob_threshold, 0.65):
0984:             for q10_floor in (-0.03, config.decision_q10_floor, -0.07):
0985:                 out = selected.copy()
0986:                 out["prob_threshold"] = prob_threshold
0987:                 out["q10_floor"] = q10_floor
0988:                 out["buy_signal"] = (out["prob_return_positive"] >= prob_threshold) &
(out["q10_predicted_net_return"] >= q10_floor)
0989:                 rows.append(out)
0990:
```

8. Snapshot Forecast và quyết định mua 1-5 tháng

Bảng dưới đây là dự báo snapshot riêng tại ngày dữ liệu cuối. Mô hình được train trên các dòng có nhãn lịch sử đã biết, sau đó dự báo cho dòng feature mới nhất. Vì tương lai sau snapshot chưa có nhãn, đây là forecast phục vụ quyết định, không phải backtest.

Cách đọc bảng snapshot như sau: expected return là lợi suất ròng kỳ vọng sau spread; q10 downside là biên dưới rủi ro ở phân vị 10%; P(return>0) là xác suất mô hình ước lượng lợi suất dương. Tín hiệu mua chỉ đáng tin hơn khi ba đại lượng này cùng ủng hộ, và kết quả backtest cùng horizon không mâu thuẫn mạnh với forecast hiện tại.

Horizon	Expected return	Q10 downside	P(return>0)	Khuyến nghị
1 tháng	-5.17%	-6.85%	0.7%	Không mua mới
3 tháng	2.77%	-8.02%	91.1%	Mua tích lũy
5 tháng	9.65%	-3.84%	100.0%	Mua tích lũy

Snapshot forecast dùng latest feature row (scripts/analysis/build_full_report.py:776-850)

Report tự tái tính snapshot forecast từ model frame và feature_columns.json để headline recommendation khớp artifact sinh ra.

```
0776: def train_snapshot_forecasts(frame: pd.DataFrame, feature_cols: list[str]) -> pd.DataFrame:
0777:     if frame.empty or not feature_cols:
0778:         return pd.DataFrame()
0779:     try:
0780:         from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
0781:         from sklearn.impute import SimpleImputer
```

```
        from sklearn.pipeline import Pipeline
    except Exception as exc:
        return pd.DataFrame([{"status": f"snapshot forecast skipped: {exc}"}])

    data = frame.copy()
    data["date"] = safe_date_series(data["date"])
    feature_cols = [c for c in feature_cols if c in data.columns]
    latest = data.sort_values("date").iloc[[-1]].copy()
    rows: list[dict[str, Any]] = []

    for horizon in [1, 3, 5]:
        target_col = f"net_return_{horizon}m"
        if target_col not in data.columns:
            continue
        train = data.dropna(subset=[target_col]).copy()
        if len(train) < 500:
            rows.append({"horizon_months": horizon, "status": "skipped_insufficient_training_rows"})
            continue

        # Select the most informative features on the training window only.
        scored: list[tuple[str, float]] = []
        y = pd.to_numeric(train[target_col], errors="coerce")
        for col in feature_cols:
            x = pd.to_numeric(train[col], errors="coerce")
            if x.notna().sum() < 200 or x.nunique(dropna=True) < 2:
                continue
            corr = abs(x.corr(y))
            if pd.notna(corr):
                scored.append((col, corr))
        selected = [col for col, _ in sorted(scored, key=lambda item: item[1], reverse=True)[:120]]
        if not selected:
            rows.append({"horizon_months": horizon, "status": "skipped_no_usable_features"})
            continue

        x_train = train[selected].apply(pd.to_numeric, errors="coerce")
        y_train = y.loc[x_train.index].astype(float)
        x_latest = latest[selected].apply(pd.to_numeric, errors="coerce")

        point_model = Pipeline(
            [
                ("imputer", SimpleImputer(strategy="median")),
                (
                    "model",
                    RandomForestRegressor(
                        n_estimators=350,
                        max_depth=7,
                        min_samples_leaf=12,
                        random_state=42,
                        n_jobs=-1,
                    ),
                ),
            ]
        )
        q10_model = Pipeline(
            [
                ("imputer", SimpleImputer(strategy="median")),
                (
                    "model",
                    GradientBoostingRegressor(
                        loss="quantile",
                        alpha=0.10,
                        n_estimators=250,
                        max_depth=3,
                        learning_rate=0.035,
                        random_state=42,
                    ),
                ),
            ]
        )
```

Raw snapshot forecast rows: bảng dưới lấy trực tiếp từ snapshot_forecasts.csv để đối chiếu với khuyến nghị ở trên.

snapshot_date	horizon_months	predicted_net_return	q10_predicted_net_return	prob_return_positive	buy_signal	status
2026-07-11	1	-0.0517143910552532	-0.0685306116065865	0.0072642105684024	False	ok
2026-07-11	3	0.0277025843487327	-0.0802303947452729	0.9108579486792878	True	ok
2026-07-11	5	0.0964738539025009	-0.0383526020847782	0.9999990714158942	True	ok

Khuyến nghị chính: có thể mua tích lũy có kiểm soát cho horizon khoảng 5 tháng, nhưng không mua mạnh một lần. Horizon 3 tháng không đủ hấp dẫn vì backtest tín hiệu âm; horizon 1 tháng chỉ phù hợp giao dịch nhỏ do tín hiệu lịch sử rất hiếm. Lưu ý: nếu mô hình snapshot 3 tháng bật tín hiệu, vẫn cần hạ trọng số nếu backtest 3 tháng có avg buy-day return âm.

Với horizon 1 tháng, forecast âm khiến quyết định mua ngắn hạn không hấp dẫn, đặc biệt khi spread làm điểm hòa vốn cao hơn. Với horizon 3 tháng, expected return gần bằng 0 nhưng q10 downside xấu, nên rủi ro không được bù đủ. Với horizon 5 tháng, forecast có tín hiệu tốt hơn, nhưng nên hiểu là mua tích lũy có kiểm soát hoặc phân bổ từng phần, không phải all-in tại một giá duy nhất.

Kết luận này phù hợp với logic kinh tế: vàng có thể vẫn là tài sản phòng vệ trong môi trường bất định, nhưng khi premium nội địa cao, người mua chịu thêm rủi ro chính sách và rủi ro co premium. Do đó, quyết định hợp lý nhất là ưu tiên quản trị vị thế, theo dõi premium/spread và cập nhật rolling forecast thay vì xem snapshot là lời khuyên cố định.

9. Caveats, Feedback và kế hoạch cải thiện

Các caveat dưới đây không phải lỗi cần che đi, mà là phần phản hồi để nâng độ tin cậy của hệ thống. Với bài toán dự báo tài chính, việc biết mô hình chưa biết gì quan trọng không kém kết quả dự báo. Mỗi caveat được chuyển thành backlog có artifact đo lường, để lần chạy sau có thể kiểm tra đã cải thiện thật hay chưa.

- Premium missing rate hiện khoảng 0.0%; kết luận premium cần đọc là directional.
- VN deposit-rate history bị loại vì value null 100%, nên chưa benchmark đầy đủ với lãi suất tiết kiệm.
- News/headline được backfill trong 2026; đã lag theo event_date nhưng real-time availability chưa chứng minh tuyệt đối.
- LightGBM/XGBoost/CatBoost đã được cài và train thật khi xuất hiện trong leaderboard; DeepAR/TFT là production-candidate cho runner riêng, không còn là blocker thiếu dependency.
- Bước tiếp theo nên ưu tiên: bổ sung LBMA/FX coverage, nguồn lãi suất tiền gửi VN, lịch đấu thầu/chính sách NHNN, rồi chạy paper-trading rolling sau snapshot.

Về premium, mục tiêu cải thiện là giảm missing nhưng không đánh đổi tính trung thực của nguồn. Vì vậy premium row phải mang theo gold_reference_source, fx_source, source_quality, availability_from và is_proxy. Những ngày dùng proxy vẫn được dùng cho mô hình nếu cần tính liên tục, nhưng report và sensitivity phải có khả năng tách high-quality-only khỏi proxy-inclusive.

Về lãi suất, báo cáo không trộn lẫn tỷ giá trung tâm SBV với deposit rate. Structure 137473 đã được xác minh là central USD/VND FX, nên không được dùng làm lãi suất tiền gửi. Khi chưa có lịch sử chính thức, hướng đúng là archive forward các bảng lãi suất ngân hàng thương mại có ngày công bố rõ, sau đó mới tạo feature opportunity cost cho các kỳ hạn 1/3/5 tháng.

Về news, chế độ research_event_date_lagged có thể dùng cho phân tích học thuật vì đã lag theo ngày bài viết, nhưng chế độ strict_realtime_verified mới phù hợp cho paper-trading. Nếu hai chế độ tạo kết quả khác nhau đáng kể, báo cáo phải hạ confidence của headline features và ưu tiên biến giá/thanh khoản có timestamp chắc chắn hơn.

10. Next Data Expansion & Paper Trading

Mục này ghi nhận phần mở rộng sau snapshot bằng dữ liệu thật: Playwright được dùng cho SBV/NHNN vì API trực tiếp có thể bị chặn; Firecrawl chỉ là fallback cho bài báo/trang tĩnh khi có API key. Không có lịch đấu thầu hay lãi suất nào được sinh giả.

Mục tiêu của phần mở rộng không phải thay đổi kết luận đầu tư một cách cơ học, mà là giảm bất định quanh các biến đang yếu: premium reference, FX, lãi suất cơ hội, policy events và khả năng kiểm chứng real-time. Khi các nguồn này tốt hơn, mô hình có thể phân biệt rõ hơn giữa một đợt tăng giá do vàng thế giới, một đợt tăng do VND mất giá, và một đợt tăng do premium nội địa bị đẩy bởi khan hiếm/chính sách.

Playwright được ưu tiên cho SBV vì phiên trình duyệt thật giữ được session và bắt được các endpoint CMS mà request trực tiếp có thể bị chặn. Collector không suy diễn lịch đấu thầu từ quy luật tuần hoặc tin đồn; chỉ event có URL chính thức, ngày công bố, hash nội dung và confidence mới được đưa vào event panel chính. Dữ liệu rule_generated nếu có chỉ được giữ như tham khảo, không đi vào mô hình chính.

Before vs After Caveats:

Caveat	Before	After	Decision impact
Premium coverage	0.0%	0.0%	Met target <10%
Deposit opportunity cost	0 non-null verified rows	15 current retail rows; SBV policy rows=0	forward_monitoring_only
News real-time availability	Backfilled event_date lagged	14.2%	Strict mode uses availability_from; backfilled rows excluded from paper-trading history.
Boosting/deep model coverage	Boosting deps previously missing	lightgbm=ok, xgboost=ok, catboost=ok, torch=ok, pytorch_forecasting=ok, gluonts=ok	Boosting models are trained when importable; deep models need a dedicated runner before inclusion.
Paper-trading status	No rolling ledger	rows=3, open=2, closed=0	Open trades have no realized return until exit price exists.

Artifact	Rows	Coverage/status	Caveat
SBV source discovery	15	137473 central FX verified	No verified SBV deposit/interest-rate content structure found; structure 137473 is central USD/VND FX.
SBV gold policy events	3	2026-07-16 to 2026-07-16	Official-event extraction completed
Rolling paper trading	3	buy=2, open=2, closed=0	No paper-trading trade has matured yet; realized performance is intentionally not calculated.
LBMA daily append	7	2026-07-16 đến 2026-07-16 (date)	Public today.json append; historical backfill still requires licensed archive or GC=F proxy.

Bảng artifact cho thấy hệ thống đã chuyển từ caveat mô tả sang caveat đo được. Premium có audit coverage; deposit rate có trạng thái forward monitoring; SBV discovery ghi rõ structure nào là tỷ giá trung tâm; paper-trading ledger ghi trạng thái open/closed thay vì tính realized return trước khi giao dịch đáo hạn. Đây là điều kiện cần để biến báo cáo từ một phân tích tĩnh thành một quy trình theo dõi sau snapshot.

SBV structures discovered from official pages/session:

Structure	Classification	Sample rows	Fields
10100	gold_policy_candidate	0	
20117	gold_policy_candidate	0	
29633	gold_policy_candidate	0	
30000	gold_policy_candidate	0	
32827	gold_policy_candidate	0	
32829	gold_policy_candidate	0	
33808	unknown	0	
137473	central_fx	10	ChuThich|NgayBanHanh|NgayBatDau|NgayKetThuc|SoVanBan|TyGiaChu|TyGiaSo

Official SBV gold-policy event types collected:

Event type	Count
gold_sale_policy	2
policy_auction	1

Paper-trading ledger status by horizon:

Horizon	Buy rows	Open	Closed	Avg realized	Hit rate
1	0	0	0	n/a	n/a
3	1	1	0	n/a	n/a
5	1	1	0	n/a	n/a

Paper-trading ledger sample: dữ liệu này nằm ngay phần monitoring để thấy horizon 5 tháng đang open và chưa có realized return.

trade_id	feature_date	horizon_months	entry_sell_price	expected_return	q10_downside	prob_positive	decision	exit_status	realized_net_return
vn_gold_20260711_1m_snapshot_forecast_v1	2026-07-11	1.0	149900000.0	-0.0517143910552532	-0.0685306116065865	0.0072642105684024	no_buy	no_position	
vn_gold_20260711_3m_snapshot_forecast_v1	2026-07-11	3.0	149900000.0	0.0277025843487327	-0.0802303947452729	0.9108579486792878	buy	open	
vn_gold_20260711_5m_snapshot_forecast_v1	2026-07-11	5.0	149900000.0	0.0964738539025009	-0.0383526020847782	0.9999990714158942	buy	open	

Paper-trading sau snapshot được thiết kế để tránh self-deception. Mỗi ngày chỉ được ghi signal dựa trên feature đã available as-of ngày quyết định; lệnh chưa đủ ngày exit phải giữ trạng thái open và không được tính vào hit rate. Khi có dữ liệu exit thật, ledger mới chuyển sang closed và cập nhật realized return sau spread. Nhờ vậy performance tương lai không bị trộn với forecast chưa đáo hạn.

Paper-trading không tính return khi lệnh chưa đáo hạn (scripts/analysis/run_rolling_paper_trading.py:105-132)

Ledger chỉ tính realized_net_return khi có exit_buy_price thật; nếu chưa tới ngày exit thì status=open.

```
0105: def build_ledger(forecasts: pd.DataFrame, prices: pd.DataFrame, model_version: str) -> pd.DataFrame:
0106:     rows: list[dict[str, Any]] = []
0107:     for _, forecast in forecasts.sort_values(["snapshot_date", "horizon_months"]).iterrows():
0108:         signal_date = forecast["snapshot_date"]
0109:         horizon_months = int(forecast["horizon_months"])
0110:         entry_row = price_on_or_before(prices, signal_date)
0111:         buy_signal = bool(forecast.get("buy_signal"))
0112:         decision = "buy" if buy_signal else "no_buy"
```

```
0113:         exit_target = signal_date + pd.DateOffset(months=horizon_months)
0114:         exit_row = price_on_or_after(prices, exit_target)
0115:
0116:         entry_sell = float(entry_row["sell"]) if entry_row is not None else None
0117:         entry_buy = float(entry_row["buy"]) if entry_row is not None else None
0118:         entry_date = entry_row["date"].date().isoformat() if entry_row is not None else ""
0119:         exit_buy = None
0120:         exit_date = ""
0121:         realized = None
0122:         status = "no_position"
0123:         if decision == "buy":
0124:             if entry_sell is None:
0125:                 status = "blocked_missing_entry_price"
0126:             elif exit_row is None:
0127:                 status = "open"
0128:             else:
0129:                 exit_buy = float(exit_row["buy"])
0130:                 exit_date = exit_row["date"].date().isoformat()
0131:                 realized = exit_buy / entry_sell - 1.0
0132:                 status = "closed"
```

11. Evidence từ crawl, data lake và code pipeline

Phần này bổ sung lớp minh chứng trực tiếp từ các file đã crawl, các bảng trong data lake và các đoạn code đang tạo dữ liệu/mô hình. Mục tiêu là để người đọc có thể lần ngược từ kết luận trong báo cáo về artifact cụ thể: dữ liệu nào được đọc, rule nào được áp dụng, và vì sao một số nguồn bị loại hoặc chỉ được dùng để monitoring.

11.1. Artifact audit: dữ liệu thật đã đi vào báo cáo

Artifact	Rows/Cols	Evidence đọc được	Ý nghĩa trong báo cáo
source_discovery/sbv_structures.csv	15 rows / 18 cols	{'gold_policy_candidate': 11, 'unknown': 3, 'central_fx': 1}	Kết quả Playwright discovery; xác nhận structure 137473 là central FX, không phải deposit rate.
events/sbv_gold_policy_events.csv	3 rows / 10 cols	{'gold_sale_policy': 2, 'policy_auction': 1}	Event chính sách vàng có URL SBV, published_at, raw_hash và confidence.
normalized/retail_deposit_rates.csv	15 rows / 13 cols	rate range 2.10-6.00%/năm	Nguồn lãi suất tiền gửi hiện tại để forward-monitoring; không backfill lịch sử suy diễn.
normalized/lbma_gold_spot_am_pm.csv	7 rows / 10 cols	LBMA_GOLD_AM_USD_OZ, LBMA_GOLD_PM_USD_OZ, LBMA_PALLADIUM_AM_USD_OZ, LBMA_PALLADIUM_PM_USD_OZ	LBMA today.json append; lịch sử vẫn dùng proxy khi thiếu licence.
news_availability_audit.csv	3,441 rows / 17 cols	{'backfilled_not_realtime': 2951, 'strict_realtime_verified': 490}	Tách research_event_date_lagged và strict_realtime_verified để tránh dùng news backfill như tín hiệu trading thật.
modeling/model_frame_daily.csv	5,485 rows / 783 cols	2011-07-06 to 2026-07-11, premium missing 0.0%	Frame cuối sau as-of join, feature engineering và target 1/3/5 tháng.
modeling/paper_trading_ledger.csv	3 rows / 21 cols	[{'horizon_months': 1.0, 'decision': 'no_buy', 'exit_status': 'no_position'}, {'horizon_months': 3.0, 'decision': 'buy', 'exit_status': 'open'}, {'horizon_months': 5.0, 'decisio...	Ledger sau snapshot; lệnh chưa đáo hạn giữ trạng thái open, không tính realized return.

Bảng trên là lớp evidence-level: mỗi hàng là một artifact đã được đọc từ data lake khi build báo cáo. Các số như 5,485 dòng model frame, 3 dòng snapshot forecast, 3 dòng paper-trading ledger và 3,441 dòng news audit đều là số đếm trực tiếp từ file, không phải mô tả thủ công.

11.2. Sample rows từ dữ liệu crawl và lake

SBV source discovery sample: các structure được lấy qua phiên Playwright và phân loại bằng field/title/seed hit.

content_structure_id	classification	http_status	row_count_sample	field_names	title_samples
10100	gold_policy_candidate	403	0		
20117	gold_policy_candidate	403	0		
29633	gold_policy_candidate	403	0		
30000	gold_policy_candidate	403	0		
32827	gold_policy_candidate	403	0		
32829	gold_policy_candidate	403	0		

SBV gold-policy event sample: chỉ giữ event có URL chính thức, ngày công bố, source_type, confidence và raw_hash.

event_date	published_at	event_type	severity	title	confidence	source_type
2026-07-16	2026-07-16	policy_auction	high	Giấy mời tham gia đấu thầu vàng	0.92	official_sbv_article
2026-07-16	2026-07-16	gold_sale_policy	medium	Quản lý hoạt động ngoại hối và hoạt động kinh doanh vàng	0.92	official_sbv_article
2026-07-16	2026-07-16	gold_sale_policy	medium	Quản lý hoạt động ngoại hối và hoạt động kinh doanh vàng	0.92	official_sbv_article

Retail deposit-rate sample: dữ liệu này được dùng cho forward monitoring, không được coi là lịch sử lãi suất đầy đủ.

date	bank	tenor_months	currency	rate_pct_annual	published_at	available_from	history_status
2026-07-16	vietinbank	1	VND	2.1	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	2	VND	2.1	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	3	VND	2.4	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	4	VND	2.4	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	5	VND	2.4	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	6	VND	3.5	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	7	VND	3.5	2026-07-16	2026-07-16	forward_monitoring_only
2026-07-16	vietinbank	8	VND	3.5	2026-07-16	2026-07-16	forward_monitoring_only

News availability sample: cùng một headline có thể dùng cho research mode, nhưng chỉ strict mode nếu availability_from không đi sau feature date.

crawl_date	event_date	headline	published_at	fetched_at	availability_from	feature_mode_strict
2026-07-11	2026-07-09	Giá vàng hôm	2026-07-09	2026-07-11	2026-07-11	backfilled_not_realtime

		nay 10/7/2026: Giá vàng miếng, nhẫn trơn SJC 9999 và giá vàng thế giới thứ Sáu - baonghean.vn				
2026-07-11	2026-07-10	Giá vàng hôm nay 11/7/2026 đà phục hồi bị chặn, kim loại quý đã 'chạm đáy', Trung Quốc tăng tốc gom hàng vì lý do gì?...	2026-07-10	2026-07-11	2026-07-11	strict_realtime_verified
2026-07-11	2026-07-11	Giá vàng hôm nay (trưa 11-7): Giá vàng nhẫn giảm - Báo Quân đội nhân dân	2026-07-11	2026-07-11	2026-07-11	strict_realtime_verified
2026-07-11	2026-07-11	Cập nhật giá vàng hôm nay 11/7/2026: Giá vàng 9999, nhẫn trơn SJC PNJ BTMC thứ Bảy - Báo Lâm Đồng	2026-07-11	2026-07-11	2026-07-11	strict_realtime_verified
2026-07-11	2026-07-10	Giá vàng hôm nay 11/7: Giá vàng trong nước đứng im, vàng thế giới giảm - VOV.VN	2026-07-10	2026-07-11	2026-07-11	strict_realtime_verified

Snapshot forecast sample: đây là đầu ra model tại ngày feature cuối, sau khi train trên các dòng lịch sử có nhãn.

snapshot_date	horizon_months	predicted_net_return	q10_predicted_net_return	prob_return_positive	buy_signal	status
2026-07-11	1	-0.0517143910552532	-0.0685306116065865	0.0072642105684024	False	ok
2026-07-11	3	0.0277025843487327	-0.0802303947452729	0.9108579486792878	True	ok
2026-07-11	5	0.0964738539025009	-0.0383526020847782	0.9999990714158942	True	ok

Paper-trading ledger sample: horizon 5 tháng có quyết định buy nhưng exit_status=open nên chưa có realized return.

trade_id	feature_date	horizon_months	entry_sell_price	expected_return	q10_downside	prob_positive	decision	exit_status	realized_net_return
vn_gold_20260711_1m_snapshot_forecast_v1	2026-07-11	1.0	149900000.0	-0.0517143910552532	-0.0685306116065865	0.0072642105684024	no_buy	no_position	
vn_gold_20260711_3m_snapshot_forecast_v1	2026-07-11	3.0	149900000.0	0.0277025843487327	-0.0802303947452729	0.9108579486792878	buy	open	

vn_gold_20260711_5m_snapshot_forecast_v1	2026-07-11	5.0	149900000.0	0.0964738539025009	-0.0383526020847782	0.9999990714158942	buy	open	

Model frame sample ở cuối snapshot: các cột giá, premium, spread và target tương lai cùng nằm trên một grain ngày.

date	buy_price	sell_price	spread_pct	premium	premium_pct	is_proxy	net_return_1m	net_return_3m	net_return_5m
2026-07-07	147000000.0	150000000.0	0.02	19563032.28	0.14884	1.0			
2026-07-08	145500000.0	148500000.0	0.020202020202020202	20971148.83	0.162531	1.0			
2026-07-09	146000000.0	149000000.0	0.020134228187919464	17554032.41	0.134056	1.0			
2026-07-10	146900000.0	149900000.0	0.020013342228152101	18802740.99	0.144417	1.0			
2026-07-11	146900000.0	149900000.0	0.020013342228152101	19702740.99	0.15133	1.0			

11.3. Code evidence: rule tạo target, chống leakage và model thật

Các đoạn code dưới đây được trích trực tiếp từ repository. Chúng là phần giải thích kỹ thuật cho các quyết định phương pháp trong báo cáo: target được tính theo giá mua tương lai/giá bán hiện tại, join dữ liệu ngoại sinh dùng as-of cutoff, optional boosting chỉ chạy khi dependency import được, và rule mua cần cả xác suất lẫn q10 downside.

Target lợi suất sau spread (src/gold_collectors/modeling/decision_support.py:386-430)

Đây là đoạn biến bài toán dự báo giá thành bài toán lợi suất thực thi: mua theo sell_price hiện tại và thoát theo future buy_price.

```
0386: def _add_targets(frame: pd.DataFrame, horizons: tuple[int, ...]) -> pd.DataFrame:
0387:     frame = frame.sort_values("date").reset_index(drop=True).copy()
0388:     dates = pd.to_datetime(frame["date"], errors="coerce").reset_index(drop=True)
0389:     buy = pd.to_numeric(frame["buy_price"], errors="coerce").reset_index(drop=True)
0390:     sell = pd.to_numeric(frame["sell_price"], errors="coerce").reset_index(drop=True)
0391:
0392:     for horizon in horizons:
0393:         target_dates = dates + pd.DateOffset(months=int(horizon))
0394:         exit_dates: list[pd.Timestamp | pd.NaT] = []
0395:         future_buy_values: list[float] = []
0396:         future_sell_values: list[float] = []
0397:         future_min_buy_values: list[float] = []
0398:         holding_days: list[float] = []
0399:
0400:         for idx, target_date in enumerate(target_dates):
0401:             if pd.isna(target_date):
0402:                 exit_dates.append(pd.NaT)
0403:                 future_buy_values.append(np.nan)
0404:                 future_sell_values.append(np.nan)
0405:                 future_min_buy_values.append(np.nan)
0406:                 holding_days.append(np.nan)
```

```
0407:                 continue
0408:
0409:             exit_pos = int(dates.searchsorted(target_date, side="left"))
0410:             if exit_pos >= len(frame):
0411:                 exit_dates.append(pd.NaT)
0412:                 future_buy_values.append(np.nan)
0413:                 future_sell_values.append(np.nan)
0414:                 future_min_buy_values.append(np.nan)
0415:                 holding_days.append(np.nan)
0416:                 continue
0417:
0418:             exit_dates.append(dates.iloc[exit_pos])
0419:             future_buy_values.append(float(buy.iloc[exit_pos]))
0420:             future_sell_values.append(float(sell.iloc[exit_pos]))
0421:             window = buy.iloc[idx + 1 : exit_pos + 1]
0422:             future_min_buy_values.append(float(window.min()) if not window.empty else np.nan)
0423:             holding_days.append(float((dates.iloc[exit_pos] - dates.iloc[idx]).days))
0424:
0425:         frame[f"target_date_{horizon}m"] = target_dates
0426:         frame[f"exit_date_{horizon}m"] = exit_dates
0427:         frame[f"holding_days_{horizon}m"] = holding_days
0428:         frame[f"future_buy_{horizon}m"] = future_buy_values
0429:         frame[f"future_sell_{horizon}m"] = future_sell_values
0430:         frame[f"net_return_{horizon}m"] = frame[f"future_buy_{horizon}m"] / frame["sell_price"] - 1
```

As-of join cho dữ liệu global (src/gold_collectors/modeling/decision_support.py:80-112)

Global market dùng cutoff t-1 trước khi merge_asof, giảm rủi ro dùng giá quốc tế đóng cửa sau giờ Việt Nam.

```
0080:     target = _safe_numeric(target, ["buy", "sell", "spread"])
0081:     target = target.dropna(subset=["date", "buy", "sell"])
0082:     target = target[(target["buy"] > 0) & (target["sell"] >= target["buy"])]
0083:     target = target.sort_values(["date", "timestamp_sort"]).groupby("date", as_index=False).tail(1)
0084:     target = target.rename(columns={"buy": "buy_price", "sell": "sell_price"})
0085:     target["mid_price"] = (target["buy_price"] + target["sell_price"]) / 2
0086:     target["spread_abs"] = target["sell_price"] - target["buy_price"]
0087:     target["spread_pct"] = target["spread_abs"] / target["sell_price"]
0088:     return target[["date", "buy_price", "sell_price", "mid_price", "spread_abs", "spread_pct"]]
0089:
0090:
0091: def _merge_global_asof(frame: pd.DataFrame, config: ModelingConfig) -> pd.DataFrame:
0092:     global_ref = _read_csv(_path(config, "pipeline_output_global_reference.csv"))
0093:     if global_ref.empty:
0094:         frame["global_feature_date"] = pd.NaT
0095:         return frame
0096:
0097:     global_ref["global_feature_date"] = _date_series(global_ref["date"])
0098:     numeric_cols = [
0099:         "dxy_index",
0100:         "gold_futures_close_usd_oz",
0101:         "lbma_price_usd_oz",
0102:         "oil_wti_usd_barrel",
0103:         "silver_futures_close_usd_oz",
0104:         "sp500_index",
0105:         "treasury_10y_pct",
0106:         "usd_vnd_market_rate",
0107:         "usd_vnd_mid",
0108:         "vix",
0109:     ]
0110:     global_ref = _safe_numeric(global_ref, numeric_cols)
0111:     keep = ["global_feature_date"] + [c for c in numeric_cols if c in global_ref.columns]
0112:     global_ref = global_ref[keep].sort_values("global_feature_date")
```

Deposit-rate as-of và opportunity cost (src/gold_collectors/modeling/decision_support.py:188-220)

Feature lãi suất chỉ được tạo khi có available_from, tenor và rate hợp lệ; nếu chưa có lịch sử thì không ép backfill.

```
0188:     keep_cols = ["macro_feature_date"] + [c for c in wide.columns if c != "macro_feature_date"][:25]
0189:     left = frame.sort_values("date").copy()
```

```
0190:     return pd.merge_asof(
0191:         left,
0192:         wide[keep_cols].sort_values("macro_feature_date"),
0193:         left_on="date",
0194:         right_on="macro_feature_date",
0195:         direction="backward",
0196:     )
0197:
0198:
0199: def _merge_deposit_asof(frame: pd.DataFrame, config: ModelingConfig) -> pd.DataFrame:
0200:     rates = _read_csv(_path(config, "normalized", "retail_deposit_rates.csv"))
0201:     if rates.empty:
0202:         return frame
0203:     rates = rates.copy()
0204:     rates["available_from"] = _date_series(rates.get("available_from", rates.get("date")))
0205:     rates["tenor_months"] = pd.to_numeric(rates.get("tenor_months"), errors="coerce")
0206:     rates["rate_pct_annual"] = pd.to_numeric(rates.get("rate_pct_annual"), errors="coerce")
0207:     rates = rates.dropna(subset=["available_from", "tenor_months", "rate_pct_annual"])
0208:     rates = rates[(rates["rate_pct_annual"] >= 0) & (rates["rate_pct_annual"] <= 20)]
0209:     if rates.empty:
0210:         return frame
0211:
0212:     horizon_to_tenor = {1: 1, 3: 3, 5: 5}
0213:     out = frame.sort_values("date").copy()
0214:     for horizon, tenor in horizon_to_tenor.items():
0215:         sub = (
0216:             rates[rates["tenor_months"].astype(int).eq(tenor)]
0217:             .groupby("available_from", as_index=False)["rate_pct_annual"]
0218:             .mean()
0219:             .sort_values("available_from")
0220:         )
```

Pipeline build model frame (src/gold_collectors/modeling/decision_support.py:465-490)

Thứ tự build frame thể hiện toàn bộ pipeline: target SJC, premium, global, GPR, macro, deposit, event, news, target và lagged features.

```
0465:     ]
0466:     base_cols += [c for c in frame.columns if c.startswith(("event_count", "severity_", "channel_",
"days_until_next_"))]
0467:     base_cols += [c for c in frame.columns if c.startswith("raw_news_")]
0468:     base_cols += [
0469:         c
0470:         for c in frame.columns
0471:         if c
0472:         in {
0473:             "imports_all_cif_m_usd",
0474:             "imports_direct_m_usd",
0475:             "vnindex_eop",
0476:             "cpi_monthly_index",
0477:             "ip_total_index",
0478:             "unemployment_rate_pct",
0479:         }
0480:     ]
0481:     base_cols = [c for c in dict.fromkeys(base_cols) if c in frame.columns]
0482:
0483:     feature_cols: list[str] = []
0484:     new_features: dict[str, pd.Series] = {}
0485:     for col in base_cols:
0486:         frame[col] = pd.to_numeric(frame[col], errors="coerce")
0487:         if col.startswith(("event_", "severity_", "channel_", "days_until_next_")):
0488:             feature_cols.append(col)
0489:             continue
0490:         for lag in config.lags:
```

Optional model training cho LightGBM/XGBoost/CatBoost (src/gold_collectors/modeling/decision_support.py:746-792)

Các mô hình boosting không còn là blocker dependency; nếu import được thì được đưa vào optional_models và train thật.

```
0746: def train_ml_models(
0747:     frame: pd.DataFrame,
0748:     feature_cols: list[str],
0749:     config: ModelingConfig | None = None,
0750: ) -> tuple[pd.DataFrame, pd.DataFrame, list[str]]:
0751:     config = config or ModelingConfig()
0752:     blockers: list[str] = []
0753:     try:
0754:         from sklearn.ensemble import GradientBoostingRegressor, RandomForestRegressor
0755:         from sklearn.linear_model import ElasticNet, Ridge
0756:         from sklearn.pipeline import make_pipeline
0757:         from sklearn.preprocessing import StandardScaler
0758:     except Exception as exc:  # pragma: no cover - depends on local env
0759:         return pd.DataFrame(), pd.DataFrame(), [f"scikit-learn unavailable: {exc}"]
0760:
0761:     feature_cols = _bounded_feature_subset(feature_cols)
0762:     optional_models: dict[str, Any] = {}
0763:     try:
0764:         from lightgbm import LGBMRegressor  # type: ignore
0765:
0766:         optional_models["lightgbm_mean"] = LGBMRegressor(n_estimators=300, learning_rate=0.03,
random_state=config.random_state, verbose=-1)
0767:         optional_models["lightgbm_q10"] = LGBMRegressor(
0768:             objective="quantile", alpha=0.10, n_estimators=300, learning_rate=0.03,
random_state=config.random_state, verbose=-1
0769:         )
0770:     except Exception as exc:
0771:         blockers.append(f"LightGBM unavailable; using sklearn quantile fallback: {exc}")
0772:     try:
0773:         from xgboost import XGBRegressor  # type: ignore
0774:
0775:         optional_models["xgboost_mean"] = XGBRegressor(
0776:             n_estimators=250, max_depth=3, learning_rate=0.03, subsample=0.9, colsample_bytree=0.9,
random_state=config.random_state
0777:         )
0778:     except Exception as exc:
0779:         blockers.append(f"XGBoost unavailable: {exc}")
0780:     try:
0781:         from catboost import CatBoostRegressor  # type: ignore
0782:
0783:         optional_models["catboost_mean"] = CatBoostRegressor(
0784:             iterations=300,
0785:             depth=4,
0786:             learning_rate=0.03,
0787:             loss_function="RMSE",
0788:             random_seed=config.random_state,
0789:             allow_writing_files=False,
0790:             verbose=False,
0791:         )
0792:         optional_models["catboost_q10"] = CatBoostRegressor(
```

Decision rule từ xác suất và q10 downside (src/gold_collectors/modeling/decision_support.py:960-990)

Tín hiệu mua không chỉ dựa vào expected return; rule yêu cầu P(return>0) vượt ngưỡng và q10 không thấp hơn sàn rủi ro.

```
0960:         selected = mean_preds[(mean_preds["horizon_months"].eq(horizon)) &
(mean_preds["model"].eq(model_name))].copy()
0961:         if selected.empty:
0962:             continue
0963:         residual_std = float((selected["actual_net_return"] -
selected["predicted_net_return"]).std(skipna=True))
0964:         residual_std = residual_std if np.isfinite(residual_std) and residual_std > 0 else 0.05
0965:         selected["prob_return_positive"] = 1.0 - selected["predicted_net_return"].apply(lambda x: 0.5 *
(1.0 + math.erf((0.0 - x) / (residual_std * math.sqrt(2)))))
0966:
0967:         q10 = predictions[
```

```
0968:             predictions["horizon_months"].eq(horizon)
0969:             & predictions["prediction_type"].eq("q10")
0970:             & predictions["model"].astype(str).str.contains("q10")
0971:         ].copy()
0972:         if not q10.empty:
0973:             q10 = q10.sort_values(["date", "model"]).drop_duplicates(["date", "horizon_months"],
keep="first")
0974:             selected = selected.merge(
0975:                 q10[["date", "horizon_months",
"predicted_net_return"]].rename(columns={"predicted_net_return": "q10_predicted_net_return"}),
0976:                 on=["date", "horizon_months"],
0977:                 how="left",
0978:             )
0979:         else:
0980:             selected["q10_predicted_net_return"] = selected["predicted_net_return"] - 1.2816 *
residual_std
0981:
0982:         selected["selected_model"] = model_name
0983:         for prob_threshold in (0.55, config.decision_prob_threshold, 0.65):
0984:             for q10_floor in (-0.03, config.decision_q10_floor, -0.07):
0985:                 out = selected.copy()
0986:                 out["prob_threshold"] = prob_threshold
0987:                 out["q10_floor"] = q10_floor
0988:                 out["buy_signal"] = (out["prob_return_positive"] >= prob_threshold) &
(out["q10_predicted_net_return"] >= q10_floor)
0989:                 rows.append(out)
0990:
```

11.4. Code evidence: crawl SBV, premium fallback và paper-trading

SBV discovery bằng Playwright và endpoint headless-delivery (scripts/pipeline/discover_sbv_sources.py:35-72)

Collector mở các trang SBV chính thức, tìm content-structure id, rồi fetch endpoint qua browser context thay vì request thẳng dễ bị chặn.

```
0035: SEED_URLS = [
0036:     "https://sbv.gov.vn/vi/",
0037:     "https://sbv.gov.vn/vi/bieu-do-ty-gia-trung-tam",
0038:     "https://sbv.gov.vn/vi/quan-ly-hoat-dong-ngoai-hoi-va-hoat-dong-kinh-doanh-vang",
0039: ]
0040:
0041: # Known verified structure: central USD/VND rate, not deposit rates.
0042: KNOWN_STRUCTURE_IDS = {"137473"}
0043:
0044:
0045: def content_fields(item: dict[str, Any]) -> dict[str, str]:
0046:     fields: dict[str, str] = {}
0047:     for field in item.get("contentFields", []) or []:
0048:         value = field.get("contentFieldValue") or {}
0049:         fields[str(field.get("name", ""))] = str(value.get("data", "") or "")
0050:     return fields
0051:
0052:
0053: def classify_structure(field_names: list[str], titles: list[str], seed_hits: list[str]) -> tuple[str,
str]:
0054:     haystack = " ".join(field_names + titles + seed_hits).lower()
0055:     if {"tygiaso", "tygiachu", "ngaybatdau"} & {f.lower() for f in field_names}:
0056:         return "central_fx", "verified: fields match SBV central USD/VND rate"
0057:     if any(k in haystack for k in ["lãi suất", "lai suat", "interest", "refinance", "tái cấp vốn", "liên
ngân hàng"]):
0058:         return "interest_rate_candidate", "candidate: text mentions SBV/market interest rates"
0059:     if any(k in haystack for k in ["đấu thầu vàng", "dau thau vang", "vàng miếng", "sjc", "kinh doanh
vàng"]):
0060:         return "gold_policy_candidate", "candidate: text mentions gold auction/SJC/gold policy"
0061:     if any(k in haystack for k in ["thông tư", "quyết định", "chỉ thị", "quy che", "quy chế"]):
0062:         return "policy_document_candidate", "candidate: text mentions official policy documents"
0063:     return "unknown", "not enough evidence to classify"
0064:
0065:
0066: def browser_fetch_structure(page: Any, structure_id: str, page_size: int) -> dict[str, Any]:
```

```
0067:     endpoint = f"/vi/o/headless-delivery/v1.0/content-structures/{structure_id}/structured-
contents?pageSize={page_size}&sort=datePublished:desc"
0068:     result = page.evaluate(
0069:         """
0070:         async ({endpoint}) => {
0071:           const response = await fetch(endpoint, {method: "GET"});
0072:           const text = await response.text();
```

Phân loại SBV event có source_url, confidence và raw_hash (scripts/pipeline/collect_sbv_gold_policy_events.py:180-199)

Mỗi event chính sách vàng phải có event_date/published_at, event_type, source_url, source_type, confidence và raw_hash.

```
0180:     event_type, severity = infer_event_type(hay)
0181:     published = parse_date_from_text(body) or parse_date_from_text(title) or ""
0182:     event_date = published
0183:     headline = normalize_text(candidate.get("anchor_text")) or title
0184:     if len(headline) < 10:
0185:         # Use the first body line that looks like a headline.
0186:         headline = normalize_text(body.split(".")[0])[:240] or title
0187:     confidence = 0.92 if published and status and 200 <= int(status) < 300 else 0.70
0188:     return {
0189:         "event_date": event_date,
0190:         "published_at": published,
0191:         "title": headline[:280],
0192:         "event_type": event_type,
0193:         "severity": severity,
0194:         "source_url": url,
0195:         "source_type": "official_sbv_article",
0196:         "confidence": confidence,
0197:         "raw_hash": hashlib.sha256(html.encode("utf-8", errors="replace")).hexdigest(),
0198:         "note": f"seed={candidate.get('seed_url', '')}; http_status={status}",
0199:     }
```

Premium coverage audit sau fallback có giới hạn (scripts/pipeline/improve_premium_coverage.py:257-287)

Premium fallback được đánh giá bằng missing rate theo năm/source quality và target <10%; không đạt thì ghi blocker thay vì im lặng.

```
0257:
0258:     new_missing_rate = float(out["premium"].isna().mean())
0259:     by_year = out.assign(year=pd.to_datetime(out["date"]).dt.year).groupby("year", as_index=False).agg(
0260:         rows=("date", "size"),
0261:         premium_missing=("premium", lambda s: int(pd.to_numeric(s, errors="coerce").isna().sum())),
0262:         premium_missing_rate=("premium", lambda s: float(pd.to_numeric(s,
errors="coerce").isna().mean())),
0263:         proxy_share=("is_proxy", lambda s: float(pd.Series(s).astype(bool).mean())),
0264:     )
0265:     by_quality = out.groupby("source_quality", as_index=False).agg(rows=("date", "size"))
0266:     audit = pd.concat(
0267:         [
0268:             by_year.assign(section="by_year").rename(columns={"year": "segment"}),
0269:             by_quality.assign(section="by_source_quality").rename(columns={"source_quality":
"segment"}),
0270:         ],
0271:         ignore_index=True,
0272:         sort=False,
0273:     )
0274:     summary = {
0275:         "generated_at": datetime.now(timezone.utc).isoformat(),
0276:         "rows": int(len(out)),
0277:         "old_rows": int(old_rows),
0278:         "old_premium_missing_rate": old_missing_rate,
0279:         "new_premium_missing_rate": new_missing_rate,
0280:         "target_missing_rate": args.target_missing_rate,
0281:         "target_met": bool(new_missing_rate <= args.target_missing_rate),
```

```
0282:         "gold_tolerance_days": args.gold_tolerance_days,
0283:         "fx_tolerance_days": args.fx_tolerance_days,
0284:         "source_quality_counts": out["source_quality"].value_counts(dropna=False).to_dict(),
0285:         "blockers": [] if new_missing_rate <= args.target_missing_rate else [
0286:             "Premium missing remains above target after bounded as-of fallback; inspect source gaps by
year."
0287:         ],
```

Paper-trading không tính return khi lệnh chưa đáo hạn (scripts/analysis/run_rolling_paper_trading.py:105-132)

Ledger chỉ tính realized_net_return khi có exit_buy_price thật; nếu chưa tới ngày exit thì status=open.

```
0105: def build_ledger(forecasts: pd.DataFrame, prices: pd.DataFrame, model_version: str) -> pd.DataFrame:
0106:     rows: list[dict[str, Any]] = []
0107:     for _, forecast in forecasts.sort_values(["snapshot_date", "horizon_months"]).iterrows():
0108:         signal_date = forecast["snapshot_date"]
0109:         horizon_months = int(forecast["horizon_months"])
0110:         entry_row = price_on_or_before(prices, signal_date)
0111:         buy_signal = bool(forecast.get("buy_signal"))
0112:         decision = "buy" if buy_signal else "no_buy"
0113:         exit_target = signal_date + pd.DateOffset(months=horizon_months)
0114:         exit_row = price_on_or_after(prices, exit_target)
0115:
0116:         entry_sell = float(entry_row["sell"]) if entry_row is not None else None
0117:         entry_buy = float(entry_row["buy"]) if entry_row is not None else None
0118:         entry_date = entry_row["date"].date().isoformat() if entry_row is not None else ""
0119:         exit_buy = None
0120:         exit_date = ""
0121:         realized = None
0122:         status = "no_position"
0123:         if decision == "buy":
0124:             if entry_sell is None:
0125:                 status = "blocked_missing_entry_price"
0126:             elif exit_row is None:
0127:                 status = "open"
0128:             else:
0129:                 exit_buy = float(exit_row["buy"])
0130:                 exit_date = exit_row["date"].date().isoformat()
0131:                 realized = exit_buy / entry_sell - 1.0
0132:                 status = "closed"
```

11.5. Đối chiếu code - dữ liệu - kết luận

Kết luận trong báo cáo	Artifact chứng minh	Code sinh ra/kiểm tra	Cách đọc
Premium missing giảm về mức báo cáo trong model frame	quality/premium_coverage_audit.csv; modeling/model_frame_daily.csv	scripts/pipeline/improve_premium_coverage.py: 257-287	Coverage được đo theo năm và source_quality; proxy vẫn được gắn cờ is_proxy.
Không dùng SBV structure 137473 làm deposit rate	source_discovery/sbv_structures.csv	scripts/pipeline/discover_sbv_sources.py:53-63	Structure có field central FX được phân loại central_fx, không phải interest_rate_candidate.
News strict realtime còn yếu	news_availability_audit.csv; quality/news_availability_summary.json	scripts/pipeline/audit_news_availability.py	Backfilled rows bị tách khỏi strict_realtime_verified để tránh dùng tin chưa chứng minh available.

Horizon 5 tháng là tín hiệu buy nhưng chưa có realized performance	modeling/snapshot_forecasts.csv; modeling/paper_trading_ledger.csv	scripts/analysis/run_rolling_paper_trading.py:105-132	Lệnh 5 tháng đang open; không tính hit rate/return trước target_exit_date.
Boosting models được train thật khi dependency có mặt	modeling/model_results.csv; modeling/model_sensitivity_summary.json	src/gold_collectors/modeling/decision_support.py:713-753	Dependency availability true; leaderboard có LightGBM/XGBoost/CatBoost nếu training hoàn tất.

Nhờ lớp đối chiếu này, báo cáo không chỉ là diễn giải định tính. Mỗi kết luận quan trọng đều có ba phần đi kèm: dữ liệu đầu vào hoặc output trong lake, đoạn code tạo/kiểm tra kết quả, và cách đọc để tránh hiểu quá mức. Đây là phần làm tài liệu có tính reproducible report thay vì chỉ là bài viết tóm tắt.

12. Notebook statistical evidence: bảng số liệu, code và kết quả chi tiết

Phần này đưa các bảng thống kê chính từ notebook chi tiết vào DOCX để người đọc không phải chỉ nhìn biểu đồ. Tất cả bảng dưới đây được tính lại trực tiếp từ data lake khi build report: cùng nguồn với notebooks/02_detailed_statistical_analysis.ipynb và docs/reports/notebook_statistical_results.md.

Notebook setup: đọc data lake, format số liệu và load artifact (scripts/analysis/build_detailed_statistical_notebook.py:45-165)

Đoạn code này là phần setup của notebook: đọc model frame, premium, model results, trading signals, snapshot, paper ledger, quality audit và news audit.

```
0045: import numpy as np
0046: import pandas as pd
0047: import matplotlib.pyplot as plt
0048: import seaborn as sns
0049: from IPython.display import display, Markdown
0050:
0051: warnings.filterwarnings("ignore")
0052: sns.set_theme(style="whitegrid")
0053: pd.set_option("display.max_columns", 80)
0054: pd.set_option("display.max_rows", 40)
0055: pd.set_option("display.float_format", lambda value: f"{value:,.4f}")
0056:
0057: ROOT = Path.cwd()
0058: LAKE = ROOT / "data" / "lake"
0059: MODELING = LAKE / "modeling"
0060: QUALITY = LAKE / "quality"
0061: REPORTS = ROOT / "docs" / "reports"
0062: REPORTS.mkdir(parents=True, exist_ok=True)
0063:
0064:
0065: def read_csv(path: Path, **kwargs) -> pd.DataFrame:
0066:     if not path.exists():
0067:         return pd.DataFrame()
0068:     return pd.read_csv(path, low_memory=False, **kwargs)
0069:
0070:
0071: def read_json(path: Path) -> dict:
0072:     if not path.exists():
0073:         return {}
0074:     return json.loads(path.read_text(encoding="utf-8"))
```

```
0075:
0076:
0077: def pct(value, digits=1):
0078:     if value is None or pd.isna(value):
0079:         return "n/a"
0080:     return f"{100 * float(value):.{digits}f}%"
0081:
0082:
0083: def pct_points(value, digits=2):
0084:     if value is None or pd.isna(value):
0085:         return "n/a"
0086:     return f"{float(value):.{digits}f}%"
0087:
0088:
0089: def money_m(value, digits=2):
0090:     if value is None or pd.isna(value):
0091:         return "n/a"
0092:     return f"{float(value) / 1_000_000:.{digits}f} triệu VND"
0093:
0094:
0095: def numeric(series: pd.Series) -> pd.Series:
0096:     return pd.to_numeric(series, errors="coerce")
0097:
0098:
0099: def as_pct_series(series: pd.Series) -> pd.Series:
0100:     values = numeric(series)
0101:     if values.dropna().abs().median() < 1:
0102:         return values * 100
0103:     return values
0104:
0105:
0106: def date_span(df: pd.DataFrame, candidates=("date", "event_date", "available_from", "snapshot_date",
"feature_date")):
0107:     for col in candidates:
0108:         if col in df.columns:
0109:             values = pd.to_datetime(df[col], errors="coerce").dropna()
0110:             if not values.empty:
0111:                 return f"{values.min().date()} -> {values.max().date()} ({col})"
0112:     return "n/a"
0113:
0114:
0115: def describe_numeric(df: pd.DataFrame, cols: list[str], scale=1.0) -> pd.DataFrame:
0116:     rows = []
0117:     for col in cols:
0118:         if col not in df.columns:
0119:             continue
0120:         values = numeric(df[col]) * scale
0121:         valid = values.dropna()
0122:         if valid.empty:
0123:             rows.append({"field": col, "n": 0, "missing_rate": 1.0})
0124:             continue
0125:         rows.append(
0126:             {
0127:                 "field": col,
0128:                 "n": int(valid.count()),
0129:                 "missing_rate": float(values.isna().mean()),
0130:                 "mean": valid.mean(),
0131:                 "std": valid.std(),
0132:                 "min": valid.min(),
0133:                 "p10": valid.quantile(0.10),
0134:                 "median": valid.median(),
0135:                 "p90": valid.quantile(0.90),
0136:                 "max": valid.max(),
0137:             }
0138:         )
0139:     return pd.DataFrame(rows)
0140:
0141:
0142: summary = read_json(MODELING / "analysis_summary.json")
0143: paper_summary = read_json(MODELING / "paper_trading_summary.json")
0144: model_frame = read_csv(MODELING / "model_frame_daily.csv", parse_dates=["date"])
0145: premium = read_csv(LAKE / "pipeline_output_premium_enriched.csv", parse_dates=["date",
"availability_from"])
```

```
model_results = read_csv(MODELING / "model_results.csv")
trading_signals = read_csv(MODELING / "trading_signals.csv", parse_dates=["date"])
snapshot = read_csv(MODELING / "snapshot_forecasts.csv", parse_dates=["snapshot_date"])
paper_ledger = read_csv(MODELING / "paper_trading_ledger.csv", parse_dates=["feature_date",
"signal_date", "target_exit_date", "exit_date"])
quality_findings = read_csv(QUALITY / "full_data_quality_findings.csv")
premium_audit = read_csv(QUALITY / "premium_coverage_audit.csv")
news_audit = read_csv(LAKE / "news_availability_audit.csv", parse_dates=["crawl_date", "event_date",
"published_at", "fetched_at", "availability_from"])
event_regime = read_csv(LAKE / "pipeline_output_event_regime.csv", parse_dates=["event_date"])
sbv_events = read_csv(LAKE / "events" / "sbv_gold_policy_events.csv", parse_dates=["event_date",
"published_at"])

target_non_null = summary.get("target_non_null", {})
critical_findings = int((quality_findings.get("severity", pd.Series(dtype=str)).astype(str).str.lower()
== "critical").sum()) if not quality_findings.empty else 0
snapshot_signal_text = ", ".join(
    str(int(row.horizon_months)) + "m=" + ("buy" if bool(row.buy_signal) else "no_buy")
    for _, row in snapshot.iterrows()
) if not snapshot.empty else "n/a"

headline = f"""
- Snapshot phân tích kết thúc tại **{summary.get('date_max', 'n/a')}** với **{summary.get('rows',
len(model_frame)):,} dòng model frame**.
- Premium missing rate: **{pct(summary.get('premium_missing_rate'))}**; premium median trong frame:
**{pct_points(as_pct_series(model_frame['premium_pct']).median()) if 'premium_pct' in model_frame else
'n/a'}**.
```

12.1. Inventory, grain và missingness

Artifact	Path	Rows	Cols	Date span	Missing	Role
model_frame_daily	data/lake/modeling/model_frame_daily.csv	5,485	783	2011-07-06 đến 2026-07-11 (date)	7.6%	Daily modeling frame
premium_enriched	data/lake/pipeline_output_premium_enriched.csv	5,485	24	2011-07-06 đến 2026-07-11 (date)	0.0%	Premium decomposition and source quality
model_results	data/lake/modeling/model_results.csv	186	8	Không có cột ngày chuẩn	22.2%	Walk-forward model metrics
trading_signals	data/lake/modeling/trading_signals.csv	3,065	17	2023-01-01 đến 2026-04-11 (date)	0.0%	Decision-rule backtest signals
snapshot_forecasts	data/lake/modeling/snapshot_forecasts.csv	3	13	Không có cột ngày chuẩn	0.0%	Latest forecast rows
paper_trading_ledger	data/lake/modeling/paper_trading_ledger.csv	3	21	Không có cột ngày chuẩn	14.3%	Post-snapshot monitoring ledger
quality_findings	data/lake/quality/full_data_quality_findings.csv	10	7	Không có cột ngày chuẩn	0.0%	Full data quality findings
premium_coverage_audit	data/lake/quality/premium_coverage_audit.csv	18	6	Không có cột ngày chuẩn	5.6%	Premium coverage by segment
news_availability_audit	data/lake/news_availability_audit.csv	3,441	17	2010-11-01 đến 2026-07-11 (event_date)	0.9%	News realtime/backfill audit

Artifact	Expected grain key	Rows	Duplicate rows	Unique keys
model_frame_daily	date	5,485	0	5,485
premium_enriched	date	5,485	0	5,485
snapshot_forecasts	snapshot_date, horizon_months	3	0	3
paper_trading_ledger	trade_id	3	0	3

Field	Non-null	Missing rate
buy_price	5,485	0.0%
sell_price	5,485	0.0%
spread_pct	5,485	0.0%
global_gold_vnd_per_luong	5,485	0.0%
premium_pct	5,485	0.0%
usd_vnd	5,485	0.0%
net_return_1m	5,455	0.5%
net_return_3m	5,394	1.7%
net_return_5m	5,335	2.7%

Inventory cho thấy notebook/report đang dùng cùng một model frame 5,485 dòng và các artifact modeling đã sinh thật. Grain check giúp kiểm tra rằng frame cuối và premium đều unique theo ngày, còn snapshot unique theo ngày snapshot và horizon tháng.

12.2. Domestic price, spread và premium: thống kê số cụ thể

Notebook EDA: thống kê giá, premium, spread và target (scripts/analysis/build_detailed_statistical_notebook.py:322-515)

Các bảng dưới đây dùng cùng logic describe_numeric, yearly aggregation, outlier p99 và target distribution trong notebook.

```
0322:     ax.plot(plot_df["date"], plot_df["global_gold_vnd_per_luong"] / 1_000_000, label="Global fair value",
linewidth=1.0, alpha=0.8)
0323: ax.set_title("SJC sell price vs global fair value")
0324: ax.set_ylabel("Million VND per lượng")
0325: ax.legend()
0326: plt.show()
0327: '''
0328:         ),
0329:         code(
0330:             r'''
0331: prem = premium.copy()
0332: prem["premium_pct_points"] = as_pct_series(prem["premium_pct"]) if "premium_pct" in prem else np.nan
0333: premium_overall = describe_numeric(prem, ["premium", "premium_pct_points", "global_gold_vnd_per_luong",
"usd_vnd"])
0334: premium_positive_share = float((prem["premium_pct_points"] > 0).mean()) if "premium_pct_points" in prem
else np.nan
0335: proxy_share = float(prem["is_proxy"].astype(str).str.lower().isin(["true", "1"]).mean()) if "is_proxy" in
prem else np.nan
0336: source_quality =
prem["source_quality"].fillna("unknown").value_counts().rename_axis("source_quality").reset_index(name="rows") if
"source_quality" in prem else pd.DataFrame()
0337:
0338: display(Markdown(f"### Premium toàn kỳ: positive share={pct(premium_positive_share)}, proxy
share={pct(proxy_share)}"))
0339: display(premium_overall.style.format({"missing_rate": "{:.1%}", "mean": "{:,.4f}", "std": "{:,.4f}", "min":
"{:,.4f}", "p10": "{:,.4f}", "median": "{:,.4f}", "p90": "{:,.4f}", "max": "{:,.4f}"}))
0340: if not source_quality.empty:
0341:     display(Markdown("### Premium source quality counts"))
0342:     display(source_quality)
0343:
0344: premium_by_year = (
0345:     prem.assign(year=prem["date"].dt.year)
0346:     .groupby("year")
```

```
0347:     .agg(
0348:         observations=("premium_pct_points", "count"),
0349:         premium_median_pct=("premium_pct_points", "median"),
0350:         premium_mean_pct=("premium_pct_points", "mean"),
0351:         premium_p10_pct=("premium_pct_points", lambda s: numeric(s).quantile(0.10)),
0352:         premium_p90_pct=("premium_pct_points", lambda s: numeric(s).quantile(0.90)),
0353:         positive_share=("premium_pct_points", lambda s: float((numeric(s) > 0).mean())),
0354:         proxy_share=("is_proxy", lambda s: float(s.astype(str).str.lower().isin(["true", "1"]).mean()) if
len(s) else np.nan),
0355:     )
0356:     .reset_index()
0357: )
0358: display(Markdown("### Premium by year: latest 12 years"))
0359: display(premium_by_year.tail(12).style.format({"premium_median_pct": "{:.2f}%", "premium_mean_pct":
"{:.2f}%", "premium_p10_pct": "{:.2f}%", "premium_p90_pct": "{:.2f}%", "positive_share": "{:.1%}", "proxy_share":
"{:.1%}"}))
0360:
0361: fig, axes = plt.subplots(1, 2, figsize=(13, 4))
0362: axes[0].plot(prem["date"], prem["premium_pct_points"], linewidth=0.8, color="#D28E00")
0363: axes[0].axhline(0, color="black", linewidth=0.8)
0364: axes[0].set_title("Premium % over time")
0365: axes[0].set_ylabel("Premium (%)")
0366: if not source_quality.empty:
0367:     axes[1].bar(source_quality["source_quality"].astype(str), source_quality["rows"], color="#8064A2")
0368:     axes[1].tick_params(axis="x", rotation=25)
0369:     axes[1].set_title("Premium source quality")
0370:     axes[1].set_ylabel("Rows")
0371: plt.tight_layout()
0372: plt.show()
0373: '''
0374:         ),
0375:         code(
0376:             r'''
0377: spread = model_frame[["date", "buy_price", "sell_price", "spread_abs", "spread_pct"]].copy()
0378: spread["spread_pct_points"] = as_pct_series(spread["spread_pct"])
0379: spread_threshold = spread["spread_pct_points"].quantile(0.99)
0380: spread_outliers = spread[spread["spread_pct_points"] >= spread_threshold].sort_values("spread_pct_points",
ascending=False)
0381:
0382: display(Markdown(f"### Spread outliers: p99 threshold = {spread_threshold:.2f}%"))
0383: display(spread_outliers.head(15).style.format({"buy_price": "{:,.0f}", "sell_price": "{:,.0f}",
"spread_abs": "{:,.0f}", "spread_pct": "{:.4f}", "spread_pct_points": "{:.2f}%"}))
0384:
0385: fig, ax = plt.subplots(figsize=(11, 4))
0386: ax.plot(spread["date"], spread["spread_pct_points"], linewidth=0.8, color="#4472C4")
0387: ax.axhline(spread_threshold, color="#C00000", linestyle="--", linewidth=0.9, label="p99")
0388: ax.set_title("SJC spread as liquidity / transaction-cost proxy")
0389: ax.set_ylabel("Spread (%)")
0390: ax.legend()
0391: plt.show()
0392: '''
0393:         ),
0394:         code(
0395:             r'''
0396: global_cols = ["global_gold_usd_oz", "usd_vnd", "global_gold_vnd_per_luong", "gold_futures_close_usd_oz",
"vix", "dxy_index", "treasury_10y_pct"]
0397: global_stats = describe_numeric(model_frame, global_cols)
0398: display(Markdown("### Global gold, FX and macro proxy stats"))
0399: display(global_stats.style.format({"missing_rate": "{:.1%}", "mean": "{:,.4f}", "std": "{:,.4f}", "min":
"{:,.4f}", "p10": "{:,.4f}", "median": "{:,.4f}", "p90": "{:,.4f}", "max": "{:,.4f}"}))
0400:
0401: target_rows = []
0402: for horizon in [1, 3, 5]:
0403:     ret_col = f"net_return_{horizon}m"
0404:     hold_col = f"holding_days_{horizon}m"
0405:     exit_col = f"exit_date_{horizon}m"
0406:     target_col = f"target_date_{horizon}m"
0407:     values = numeric(model_frame[ret_col]) if ret_col in model_frame else pd.Series(dtype=float)
0408:     holds = numeric(model_frame[hold_col]) if hold_col in model_frame else pd.Series(dtype=float)
0409:     target_rows.append(
0410:         {
0411:             "horizon_months": horizon,
0412:             "target_non_null": int(values.notna().sum()),
```

```
            "positive_rate": float((values.dropna() > 0).mean()) if values.notna().any() else np.nan,
            "mean_return": values.mean(),
            "p10_return": values.quantile(0.10),
            "median_return": values.median(),
            "p90_return": values.quantile(0.90),
            "holding_days_median": holds.median(),
            "holding_days_min": holds.min(),
            "holding_days_max": holds.max(),
            "exit_date_ge_target": bool((pd.to_datetime(model_frame[exit_col], errors="coerce") >=
pd.to_datetime(model_frame[target_col], errors="coerce")).dropna().all()) if exit_col in model_frame and
target_col in model_frame else None,
        }
    )
target_stats = pd.DataFrame(target_rows)
display(Markdown("### Target returns and holding-days by calendar horizon"))
display(target_stats.style.format({"positive_rate": "{:.1%}", "mean_return": "{:.2%}", "p10_return":
"{:.2%}", "median_return": "{:.2%}", "p90_return": "{:.2%}", "holding_days_median": "{:.0f}", "holding_days_min":
"{:.0f}", "holding_days_max": "{:.0f}"}))

plot_targets = []
hold_targets = []
for horizon in [1, 3, 5]:
    ret_col = f"net_return_{horizon}m"
    hold_col = f"holding_days_{horizon}m"
    if ret_col in model_frame:
        plot_targets.append(pd.DataFrame({"horizon": f"{horizon}m", "target_return_pct":
numeric(model_frame[ret_col]) * 100}))
    if hold_col in model_frame:
        hold_targets.append(pd.DataFrame({"horizon": f"{horizon}m", "holding_days":
numeric(model_frame[hold_col])}))
plot_targets = pd.concat(plot_targets, ignore_index=True).dropna()
hold_targets = pd.concat(hold_targets, ignore_index=True).dropna()
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.boxplot(data=plot_targets, x="horizon", y="target_return_pct", ax=axes[0], showfliers=False,
color="#F4B183")
axes[0].axhline(0, color="black", linewidth=0.8)
axes[0].set_title("Target return distribution")
axes[0].set_ylabel("Net return after spread (%)")
sns.boxplot(data=hold_targets, x="horizon", y="holding_days", ax=axes[1], color="#A9D18E")
axes[1].set_title("Calendar target holding days")
plt.tight_layout()
plt.show()
'''
        ),
        code(
            r'''
exclude_prefixes = ("target_date_", "exit_date_", "future_", "net_return_")
numeric_cols = []
for col in model_frame.columns:
    if col == "date" or col.startswith(exclude_prefixes):
        continue
    values = numeric(model_frame[col])
    if values.notna().sum() >= 500 and values.nunique(dropna=True) > 2:
        numeric_cols.append(col)

corr_rows = []
for horizon in [1, 3, 5]:
    target_col = f"net_return_{horizon}m"
    if target_col not in model_frame:
        continue
    y = numeric(model_frame[target_col])
    for col in numeric_cols:
        x = numeric(model_frame[col])
        valid = x.notna() & y.notna()
        if valid.sum() < 500:
            continue
        corr = x[valid].corr(y[valid])
        if pd.notna(corr):
            corr_rows.append({"horizon_months": horizon, "feature": col, "correlation": corr,
"abs_correlation": abs(corr), "n": int(valid.sum())})
correlations = pd.DataFrame(corr_rows)
top_corr = correlations.sort_values(["horizon_months", "abs_correlation"], ascending=[True,
False]).groupby("horizon_months").head(15)
```

```
0477: display(Markdown("### Top absolute feature-target correlations by horizon"))
0478: display(top_corr[["horizon_months", "feature", "correlation", "abs_correlation",
"n"]].style.format({"correlation": "{:.4f}", "abs_correlation": "{:.4f}"}))
0479: '''
0480:         ),
0481:         code(
0482:             r'''
0483: news_summary_rows = []
0484: if not news_audit.empty:
0485:     strict_share =
float(news_audit["feature_mode_strict"].astype(str).eq("strict_realtime_verified").mean()) if
"feature_mode_strict" in news_audit else np.nan
0486:     lag = numeric(news_audit["backfill_lag_days"]) if "backfill_lag_days" in news_audit else
pd.Series(dtype=float)
0487:     news_summary_rows.append({"rows": len(news_audit), "strict_realtime_verified_share": strict_share,
"backfill_lag_median_days": lag.median(), "backfill_lag_p90_days": lag.quantile(0.90), "date_span":
date_span(news_audit, ("event_date",))})
0488: news_summary = pd.DataFrame(news_summary_rows)
0489: display(Markdown("### News availability summary"))
0490: display(news_summary.style.format({"strict_realtime_verified_share": "{:.1%}", "backfill_lag_median_days":
"{:.0f}", "backfill_lag_p90_days": "{:.0f}"}))
0491:
0492: if "category" in news_audit:
0493:     display(Markdown("### News category counts"))
0494:
display(news_audit["category"].fillna("unknown").value_counts().head(12).rename_axis("category").reset_index(name
="rows"))
0495: if "event_type" in event_regime:
0496:     display(Markdown("### Event-regime type counts"))
0497:
display(event_regime["event_type"].fillna("unknown").value_counts().head(15).rename_axis("event_type").reset_inde
x(name="rows"))
0498: if "event_type" in sbv_events:
0499:     display(Markdown("### Official SBV gold-policy event counts"))
0500:
display(sbv_events["event_type"].fillna("unknown").value_counts().rename_axis("event_type").reset_index(name="row
s"))
0501:
0502: fig, axes = plt.subplots(1, 2, figsize=(12, 4))
0503: if "backfill_lag_days" in news_audit:
0504:     lag_clip =
numeric(news_audit["backfill_lag_days"]).dropna().clip(upper=numeric(news_audit["backfill_lag_days"]).quantile(0.
99))
0505:     axes[0].hist(lag_clip, bins=40, color="#70AD47")
0506:     axes[0].set_title("News backfill lag distribution")
0507:     axes[0].set_xlabel("Lag days, clipped at p99")
0508: if "event_type" in event_regime:
0509:     top_events = event_regime["event_type"].fillna("unknown").value_counts().head(8)
0510:     axes[1].barh(top_events.index.astype(str), top_events.values, color="#5B9BD5")
0511:     axes[1].invert_yaxis()
0512:     axes[1].set_title("Top event types")
0513: plt.tight_layout()
0514: plt.show()
0515: '''
```

Domestic price/spread summary toàn kỳ:

Field	N	Missing	Mean	Std	P10	Median	P90	Max
buy_price	5,485	0.0%	56,553,470.374	31,103,102.328	35,164,000.000	43,800,000.000	87,000,000.000	187,900,000.000
sell_price	5,485	0.0%	57,248,918.870	31,792,848.323	35,270,000.000	43,950,000.000	89,300,000.000	190,900,000.000
mid_price	5,485	0.0%	56,901,194.622	31,447,322.730	35,217,000.000	43,860,000.000	88,000,000.000	189,400,000.000
spread_abs	5,485	0.0%	695,448.496	799,956.472	120,000.000	300,000.000	2,000,000.000	5,000,000.000
spread_pct	5,485	0.0%	0.99%	0.73%	0.33%	0.75%	2.08%	6.92%

Year	Obs	Sell mean	Sell median	Sell min	Sell max	Spread median	Spread P90
2015	365	34.44 triệu	34.55 triệu	32.70 triệu	35.82 triệu	0.34%	0.85%
2016	366	35.00 triệu	35.48 triệu	32.83 triệu	39.70 triệu	0.75%	1.42%
2017	365	36.62 triệu	36.60 triệu	36.10 triệu	37.75 triệu	0.55%	0.82%
2018	365	36.76 triệu	36.75 triệu	36.38 triệu	37.17 triệu	0.49%	0.54%
2019	365	39.15 triệu	39.05 triệu	36.24 triệu	42.90 triệu	0.56%	0.82%
2020	366	51.33 triệu	49.74 triệu	42.64 triệu	62.40 triệu	0.92%	2.08%
2021	365	57.41 triệu	57.10 triệu	54.50 triệu	61.80 triệu	1.06%	1.22%
2022	365	67.15 triệu	67.20 triệu	61.45 triệu	73.50 triệu	1.20%	1.52%
2023	365	68.54 triệu	67.30 triệu	66.55 triệu	79.50 triệu	1.02%	1.35%
2024	366	81.87 triệu	81.00 triệu	74.00 triệu	92.40 triệu	2.52%	3.27%
2025	365	121.56 triệu	121.10 triệu	84.20 triệu	159.70 triệu	1.65%	2.51%
2026	192	167.05 triệu	167.50 triệu	138.30 triệu	190.90 triệu	1.73%	2.04%

Premium summary toàn kỳ: positive share 95.6%, proxy share 100.0%.

Field	N	Missing	Mean	Std	P10	Median	P90	Max
premium	5,485	0.0%	6,314,416.664	6,179,082.695	472,819.690	4,014,259.880	16,764,596.738	42,117,647.240
premium_pct_points	5,485	0.0%	11.64%	9.04%	1.20%	10.21%	24.22%	43.58%
global_gold_vnd_per_luong	5,485	0.0%	50,914,064.649	27,268,994.278	31,429,690.522	41,703,406.690	83,591,867.132	167,020,083.750
usd_vnd	5,485	0.0%	22,848.771	1,633.253	20,810.000	22,825.000	25,450.000	26,425.000

Source quality	Rows
proxy_exact	3769
proxy_forward_filled	1716

Year	Obs	Premium median	Premium mean	Premium P10	Premium P90	Positive share	Proxy share
2015	365	14.21%	14.07%	10.13%	17.45%	100.0%	100.0%
2016	366	3.85%	5.84%	0.99%	13.73%	97.0%	100.0%
2017	365	7.13%	8.04%	4.63%	13.37%	100.0%	100.0%
2018	365	4.50%	4.88%	1.00%	9.20%	97.8%	100.0%
2019	365	0.68%	0.58%	-1.04%	2.21%	68.5%	100.0%
2020	366	3.18%	3.33%	-0.57%	7.70%	80.6%	100.0%
2021	365	15.55%	15.62%	9.53%	22.86%	100.0%	100.0%
2022	365	33.13%	32.35%	24.53%	38.93%	100.0%	100.0%
2023	365	22.36%	22.81%	19.06%	27.56%	100.0%	100.0%
2024	366	8.49%	14.26%	4.52%	28.23%	100.0%	100.0%
2025	365	13.61%	11.83%	3.64%	17.48%	100.0%	100.0%
2026	192	13.46%	13.73%	9.85%	18.19%	100.0%	100.0%

Spread outliers: ngưỡng P99 của spread_pct là 3.30%; bảng dưới là 15 ngày spread rộng nhất.

Date	Buy	Sell	Spread abs	Spread %
2020-08-12	51.36 triệu	55.18 triệu	3.82 triệu	6.92%
2024-11-07	81.00 triệu	85.50 triệu	4.50 triệu	5.26%
2020-08-13	53.18 triệu	55.78 triệu	2.60 triệu	4.66%

2024-11-08	82.00 triệu	86.00 triệu	4.00 triệu	4.65%
2024-05-31	83.00 triệu	87.00 triệu	4.00 triệu	4.60%
2024-11-09	82.00 triệu	85.80 triệu	3.80 triệu	4.43%
2024-11-10	82.00 triệu	85.80 triệu	3.80 triệu	4.43%
2024-11-15	80.00 triệu	83.50 triệu	3.50 triệu	4.19%
2024-11-16	80.00 triệu	83.50 triệu	3.50 triệu	4.19%
2024-11-14	80.00 triệu	83.50 triệu	3.50 triệu	4.19%
2024-11-17	80.00 triệu	83.50 triệu	3.50 triệu	4.19%
2024-11-12	80.50 triệu	84.00 triệu	3.50 triệu	4.17%
2024-11-13	80.50 triệu	84.00 triệu	3.50 triệu	4.17%
2024-11-11	81.90 triệu	85.40 triệu	3.50 triệu	4.10%
2023-12-30	71.00 triệu	74.00 triệu	3.00 triệu	4.05%

12.3. Target tháng lịch, tương quan feature và biến sự kiện/news

Horizon	Target N	Positive rate	Mean return	P10	Median	P90	Hold median	Hold min	Hold max	Exit>=target
1m	5,455	35.1%	-0.18%	-3.21%	-0.62%	3.57%	31	28	31	False
3m	5,394	47.9%	1.43%	-5.77%	-0.15%	10.88%	92	89	92	False
5m	5,335	51.9%	3.25%	-5.98%	0.22%	15.16%	153	150	153	False

Horizon	Feature	Correlation	Abs corr	N
1m	gross_sell_return_1m	0.9827	0.9827	5,455
1m	gross_sell_return_3m	0.5713	0.5713	5,394
1m	gross_sell_return_5m	0.3897	0.3897	5,335
1m	raw_news_gold_vn_count_ma30_lag_105d	-0.2551	0.2551	5,350
1m	raw_news_gold_vn_count_ma30_lag_63d	-0.2262	0.2262	5,392
1m	premium_pct_roll_std_63d	-0.2242	0.2242	5,435
1m	raw_news_sentiment_balance_ma30_lag_105d	-0.2154	0.2154	5,350
1m	raw_news_gold_vn_count_ma7_roll_mean_63d	-0.2017	0.2017	5,435
1m	raw_news_count_ma30_lag_105d	-0.1897	0.1897	5,350
1m	raw_news_gold_vn_count_ma7_lag_21d	-0.1832	0.1832	5,434
3m	gross_sell_return_3m	0.9951	0.9951	5,394
3m	gross_sell_return_5m	0.7929	0.7929	5,335
3m	gross_sell_return_1m	0.6158	0.6158	5,394
3m	usd_vnd_lag_63d	0.4153	0.4153	5,331
3m	usd_vnd_roll_mean_63d	0.4072	0.4072	5,374
3m	usd_vnd_lag_21d	0.4046	0.4046	5,373
3m	usd_vnd_mid_roll_mean_63d	0.4046	0.4046	5,373
3m	usd_vnd_lag_105d	0.4020	0.4020	5,289
3m	usd_vnd_lag_10d	0.3922	0.3922	5,384
3m	usd_vnd_roll_mean_21d	0.3909	0.3909	5,388
5m	gross_sell_return_5m	0.9974	0.9974	5,335
5m	gross_sell_return_3m	0.8074	0.8074	5,335
5m	usd_vnd_roll_mean_63d	0.5753	0.5753	5,315
5m	usd_vnd_lag_21d	0.5737	0.5737	5,314
5m	usd_vnd_mid_roll_mean_63d	0.5729	0.5729	5,314
5m	usd_vnd_lag_63d	0.5704	0.5704	5,272
5m	usd_vnd_lag_105d	0.5699	0.5699	5,230
5m	usd_vnd_lag_10d	0.5675	0.5675	5,325
5m	usd_vnd_roll_mean_21d	0.5672	0.5672	5,329
5m	usd_vnd_lag_5d	0.5644	0.5644	5,330

Bảng tương quan là diagnostic, không phải bằng chứng nhân quả. Nó cho biết feature nào đáng chú ý cho từng horizon trước khi đọc model leaderboard.

Rows	Strict realtime share	Backfill lag median	Backfill lag P90	Date span
3,441	14.2%	31	788	2010-11-01 đến 2026-07-11 (event_date)

News category	Rows
gold_vn	2520
other	626
fx_vnd	218
policy_rates	50
equity	15
macro	12

Event type	Rows
wedding_season	1530
tet_proximity	240
geopolitical_crisis	21
than_tai	16
eurozone_crisis	12
financial_stress	8
policy_import	6
financial_crisis	5
banking_stress	4
policy_inspection	4
policy_rate_increase	2
policy_auction	1
policy_rate_decrease	1

Official SBV event type	Rows
gold_sale_policy	2
policy_auction	1

12.4. Model, signal, snapshot và paper-trading: bảng kết quả cụ thể

Notebook modeling outputs: leaderboard, signals, snapshot, paper ledger (scripts/analysis/build_detailed_statistical_notebook.py:556-700)

Đoạn code này tạo các bảng model performance, decision signal statistics, snapshot forecast và paper-trading status trong notebook.

```
0556: plt.show()
0557: '''
0558:         ),
0559:         code(
0560:             r'''
0561: if not trading_signals.empty:
0562:     threshold_counts = trading_signals[["prob_threshold",
"q10_floor"]].value_counts().reset_index(name="rows").sort_values("rows", ascending=False)
0563:     default_prob = threshold_counts.iloc[0]["prob_threshold"]
0564:     default_q10 = threshold_counts.iloc[0]["q10_floor"]
0565:     default_signals = trading_signals[(trading_signals["prob_threshold"].eq(default_prob)) &
(trading_signals["q10_floor"].eq(default_q10))].copy()
0566:     default_signals["buy_signal_any"] =
default_signals["buy_signal_any"].astype(str).str.lower().isin(["true", "1"])
0567:     signal_stats = default_signals.groupby(["horizon_months", "phase"], as_index=False).agg(
0568:         observations=("date", "count"),
0569:         signal_days=("buy_signal_any", "sum"),
0570:         signal_rate=("buy_signal_any", "mean"),
0571:         avg_actual_return=("avg_actual_return", "mean"),
```

```
0572:         avg_strategy_return=("strategy_return", "mean"),
0573:         avg_probability_positive=("prob_positive", "mean"),
0574:     )
0575:     display(Markdown(f"### Decision-rule stats for dominant threshold pair: P>={default_prob},
q10>={default_q10}"))
0576:     display(signal_stats.style.format({"signal_rate": "{:.1%}", "avg_actual_return": "{:.2%}",
"avg_strategy_return": "{:.2%}", "avg_probability_positive": "{:.1%}"}))
0577:
0578:     fig, axes = plt.subplots(1, 2, figsize=(12, 4))
0579:     signal_stats["label"] = signal_stats["phase"].astype(str) + " " +
signal_stats["horizon_months"].astype(str) + "m"
0580:     axes[0].bar(signal_stats["label"], signal_stats["signal_rate"] * 100, color="#70AD47")
0581:     axes[0].set_title("Signal rate")
0582:     axes[0].set_ylabel("Signal days (%)")
0583:     axes[0].tick_params(axis="x", rotation=35)
0584:     colors = ["#4472C4" if value >= 0 else "#C00000" for value in signal_stats["avg_actual_return"]]
0585:     axes[1].bar(signal_stats["label"], signal_stats["avg_actual_return"] * 100, color=colors)
0586:     axes[1].axhline(0, color="black", linewidth=0.8)
0587:     axes[1].set_title("Average target return on signal rows")
0588:     axes[1].set_ylabel("Avg return (%)")
0589:     axes[1].tick_params(axis="x", rotation=35)
0590:     plt.tight_layout()
0591:     plt.show()
0592: '''
0593:         ),
0594:         code(
0595:             r'''
0596: display(Markdown("### Snapshot forecast rows"))
0597: if not snapshot.empty:
0598:     display(snapshot[["snapshot_date", "horizon_months", "point_model", "q10_model", "train_rows",
"feature_count", "predicted_net_return", "q10_predicted_net_return", "prob_return_positive", "buy_signal",
"status"]].style.format({"predicted_net_return": "{:.2%}", "q10_predicted_net_return": "{:.2%}",
"prob_return_positive": "{:.2%}"}))
0599:
0600: paper_status = paper_ledger.groupby(["horizon_months", "decision", "exit_status"], as_index=False).agg(
0601:     rows=("trade_id", "count"),
0602:     expected_return_mean=("expected_return", "mean"),
0603:     q10_downside_mean=("q10_downside", "mean"),
0604:     realized_non_null=("realized_net_return", lambda s: int(s.notna().sum())),
0605: ) if not paper_ledger.empty else pd.DataFrame()
0606: display(Markdown("### Paper-trading status"))
0607: if not paper_status.empty:
0608:     display(paper_status.style.format({"expected_return_mean": "{:.2%}", "q10_downside_mean":
"{:.2%}"}))
0609:     display(paper_ledger[["trade_id", "feature_date", "horizon_months", "entry_sell_price",
"expected_return", "q10_downside", "prob_positive", "decision", "target_exit_date", "exit_status",
"realized_net_return"]].style.format({"entry_sell_price": "{:,.0f}", "expected_return": "{:.2%}",
"q10_downside": "{:.2%}", "prob_positive": "{:.2%}", "realized_net_return": "{:.2%}"}, na_rep=""))
0610:
0611: open_realized = 0
0612: if not paper_ledger.empty and {"exit_status", "realized_net_return"}.issubset(paper_ledger.columns):
0613:     open_realized =
int(paper_ledger[paper_ledger["exit_status"].astype(str).eq("open")]["realized_net_return"].notna().sum())
0614: print(f"Open trades with realized return populated: {open_realized}")
0615: assert open_realized == 0
0616: '''
0617:         ),
0618:         md(
0619:             """
0620:             ## Data Quality Findings
0621:
0622:             Các finding dưới đây là từ audit artifact, không phải nhận xét thủ công. Mục đích là đọc kết
quả model cùng với rủi ro dữ liệu còn tồn tại.
0623:             """
0624:         ),
0625:         code(
0626:             r'''
0627: if not quality_findings.empty:
0628:     severity_counts =
quality_findings["severity"].value_counts().rename_axis("severity").reset_index(name="findings")
0629:     display(Markdown("### Data quality severity counts"))
0630:     display(severity_counts)
0631:     display(Markdown("### Full data quality findings"))
```

```
0632:     display(quality_findings[["severity", "check", "status", "evidence", "recommendation",
"confidence"]])
0633:
0634: if not premium_audit.empty:
0635:     display(Markdown("### Premium coverage audit by segment"))
0636:     display(premium_audit.style.format({"premium_missing_rate": "{:.1%}", "proxy_share": "{:.1%}"}))
0637: '''
0638:         ),
0639:         md(
0640:             """
0641:             ## Takeaways
0642:
0643:             Cell cuối ghi lại kết luận bằng số liệu đã tính trong notebook và xuất thêm markdown
companion để dễ đọc ngoài Jupyter.
0644:             """
0645:         ),
0646:         code(
0647:             r'''
0648: latest_sell = numeric(model_frame.sort_values("date").tail(1)["sell_price"]).iloc[0] if not
model_frame.empty else np.nan
0649: latest_premium_pct = as_pct_series(model_frame.sort_values("date").tail(1)["premium_pct"]).iloc[0] if
"premium_pct" in model_frame else np.nan
0650: premium_median = as_pct_series(model_frame["premium_pct"]).median() if "premium_pct" in model_frame else
np.nan
0651: spread_median = as_pct_series(model_frame["spread_pct"]).median() if "spread_pct" in model_frame else
np.nan
0652: best_test = leaderboard[leaderboard["phase"].eq("test")].sort_values(["horizon_months",
"mae"]).groupby("horizon_months").head(1) if "leaderboard" in globals() else pd.DataFrame()
0653:
0654: snapshot_lines = []
0655: for _, row in snapshot.sort_values("horizon_months").iterrows() if not snapshot.empty else []:
0656:     snapshot_lines.append(f"- {int(row['horizon_months'])}m: expected={row['predicted_net_return']:.2%},
q10={row['q10_predicted_net_return']:.2%}, P+={row['prob_return_positive']:.2%}, decision={'buy' if
bool(row['buy_signal']) else 'no_buy'}")
0657:
0658: best_model_lines = []
0659: for _, row in best_test.iterrows() if not best_test.empty else []:
0660:     best_model_lines.append(f"- {int(row['horizon_months'])}m test: {row['model']} có
MAE={row['mae']:.4f}, DA={row['directional_accuracy']:.1%}")
0661:
0662: md_text = f"""
0663: # Notebook Statistical Results Summary
0664:
0665: Generated from `notebooks/02_detailed_statistical_analysis.ipynb`.
0666:
0667: ## Headline Numbers
0668:
0669: - Model frame: {len(model_frame):,} rows, {len(model_frame.columns):,} columns, date span
{date_span(model_frame)}.
0670: - Latest SJC sell price: {money_m(latest_sell)}; latest premium: {latest_premium_pct:.2f}%.
0671: - Premium median across frame: {premium_median:.2f}%; premium missing rate from summary:
{pct(summary.get('premium_missing_rate'))}.
0672: - Median spread: {spread_median:.2f}%.
0673: - Target non-null: 1m={target_non_null.get('net_return_1m', 'n/a')},
3m={target_non_null.get('net_return_3m', 'n/a')}, 5m={target_non_null.get('net_return_5m', 'n/a')}.
0674: - Data quality findings: {len(quality_findings)} total, {critical_findings} critical.
0675: - Paper-trading: rows={paper_summary.get('rows', len(paper_ledger))},
open={paper_summary.get('open_rows', 'n/a')}, closed={paper_summary.get('closed_rows', 'n/a')}.
0676:
0677: ## Snapshot Decisions
0678:
0679: {chr(10).join(snapshot_lines) if snapshot_lines else '- No snapshot forecasts available.'}
0680:
0681: ## Best Test Models
0682:
0683: {chr(10).join(best_model_lines) if best_model_lines else '- No test leaderboard available.'}
0684:
0685: ## Interpretation
0686:
0687: - Premium is materially positive after the corrected troy-ounce-to-lượng conversion, so SJC should not
be interpreted as structurally cheaper than global gold.
0688: - Calendar-month targets are explicit through target date, exit date and holding-day checks; model
results should be read as 1m/3m/5m calendar horizons.
```

```
0689: - Snapshot output supports no-buy for 1m and buy signals for 3m/5m, but paper-trading entries remain
open until their exit dates mature.
0690: - News/headline features remain caveated unless strict realtime availability is verified for the
decision date.
0691: """.strip()
0692:
0693: summary_path = REPORTS / "notebook_statistical_results.md"
0694: summary_path.write_text(md_text + "\n", encoding="utf-8")
0695: display(Markdown(md_text))
0696: print(f"Wrote {summary_path.relative_to(ROOT)}")
0697: '''
0698:         ),
0699:     ]
0700:
```

Horizon	Phase	Model	MAE	RMSE	Directional accuracy	Folds
1m	test	sarimax_exog	0.0555	0.0767	26.8%	1
1m	test	naive_zero_return	0.0587	0.0742	0.0%	2
1m	test	historical_median_return	0.0597	0.0750	45.9%	2
1m	test	historical_mean_return	0.0597	0.0752	45.9%	2
1m	test	elastic_net	0.0783	0.0976	35.1%	2
1m	validation	historical_median_return	0.0232	0.0359	61.6%	2
1m	validation	historical_mean_return	0.0234	0.0360	61.6%	2
1m	validation	naive_zero_return	0.0240	0.0364	3.0%	2
1m	validation	sarimax_exog	0.0252	0.0313	69.3%	1
1m	validation	random_forest	0.0283	0.0403	62.3%	2
3m	test	ridge	0.1434	0.1743	54.1%	2
3m	test	naive_zero_return	0.1434	0.1605	0.0%	2
3m	test	historical_median_return	0.1447	0.1616	41.3%	2
3m	test	historical_mean_return	0.1458	0.1640	58.7%	2
3m	test	sarimax_exog	0.1459	0.1606	40.5%	1
3m	validation	historical_mean_return	0.0469	0.0578	62.2%	2
3m	validation	naive_zero_return	0.0479	0.0591	1.1%	2
3m	validation	historical_median_return	0.0494	0.0612	36.7%	2
3m	validation	catboost_mean	0.0640	0.0804	49.4%	2
3m	validation	random_forest	0.0652	0.0823	44.0%	2
5m	test	sarimax_exog	0.0943	0.1234	98.4%	1
5m	test	lightgbm_mean	0.1359	0.1606	53.9%	2
5m	test	catboost_mean	0.1392	0.1543	53.9%	2
5m	test	xgboost_mean	0.1421	0.1569	53.9%	2
5m	test	random_forest	0.1519	0.1734	53.9%	2
5m	validation	historical_mean_return	0.0823	0.1139	69.8%	2
5m	validation	naive_zero_return	0.0864	0.1206	0.4%	2
5m	validation	historical_median_return	0.0877	0.1223	29.8%	2
5m	validation	random_forest	0.1072	0.1297	41.4%	2
5m	validation	lightgbm_mean	0.1090	0.1339	41.4%	2

Horizon	Phase	Quantile model	Pinball loss	Folds
1m	test	sklearn_gbr_q05	0.01574	2
1m	test	sklearn_gbr_q90	0.01976	2
1m	test	sklearn_gbr_q10	0.02309	2
1m	test	lightgbm_q10	0.02569	2
1m	test	catboost_q10	0.02639	2
1m	validation	sklearn_gbr_q90	0.00635	2
1m	validation	sklearn_gbr_q05	0.00665	2
1m	validation	sklearn_gbr_q10	0.00841	2
1m	validation	lightgbm_q10	0.00863	2
1m	validation	catboost_q10	0.00867	2
3m	test	sklearn_gbr_q90	0.03921	2

3m	test	sklearn_gbr_q05	0.04381	2
3m	test	sklearn_gbr_q10	0.06615	2
3m	test	catboost_q10	0.07810	2
3m	test	lightgbm_q10	0.08278	2
3m	validation	sklearn_gbr_q05	0.00860	2
3m	validation	sklearn_gbr_q10	0.01306	2
3m	validation	catboost_q10	0.01704	2
3m	validation	lightgbm_q10	0.01836	2
3m	validation	sklearn_gbr_q90	0.01846	2
5m	test	sklearn_gbr_q90	0.02372	2
5m	test	sklearn_gbr_q05	0.06188	2
5m	test	sklearn_gbr_q10	0.06746	2
5m	test	lightgbm_q10	0.06925	2
5m	test	catboost_q10	0.07721	2
5m	validation	sklearn_gbr_q05	0.00672	2
5m	validation	sklearn_gbr_q10	0.01236	2
5m	validation	catboost_q10	0.03180	2
5m	validation	lightgbm_q10	0.04044	2
5m	validation	sklearn_gbr_q90	0.04910	2

Decision-rule stats dùng dominant threshold pair: P>=0.5, q10>=-0.1.

Horizon	Phase	Obs	Signal days	Signal rate	Avg actual return	Avg strategy return	Avg P+
1m	test	365	32	8.8%	3.87%	0.12%	38.8%
1m	validation	365	0	0.0%	-0.25%	0.00%	27.7%
3m	test	466	328	70.4%	10.56%	6.95%	59.4%
3m	validation	731	306	41.9%	1.94%	0.61%	49.2%
5m	test	407	407	100.0%	19.95%	19.95%	95.4%
5m	validation	731	366	50.1%	6.85%	3.53%	63.6%

Snapshot date	Horizon	Point model	Q10 model	Train rows	Features	Expected	Q10	P+	Buy	Status
2026-07-11	1m	random_forest_snapshot_top120_features	gradient_boosting_quantile_q10_snapshot_top120_features	5,455	120	-5.17%	-6.85%	0.73%	False	ok
2026-07-11	3m	random_forest_snapshot_top120_features	gradient_boosting_quantile_q10_snapshot_top120_features	5,394	120	2.77%	-8.02%	91.09%	True	ok
2026-07-11	5m	random_forest_snapshot_top120_features	gradient_boosting_quantile_q10_snapshot_top120_features	5,335	120	9.65%	-3.84%	100.00%	True	ok

Horizon	Decision	Exit status	Rows	Expected mean	Q10 mean	Realized non-null
1m	no_buy	no_position	1	-5.17%	-6.85%	0
3m	buy	open	1	2.77%	-8.02%	0
5m	buy	open	1	9.65%	-3.84%	0

trade_id	feature_date	horizon_months	entry_sell_price	expected_return	q10_downside	prob_positive	decision	target_exit_date	exit_status	realized_net_return
vn_gold_20260711_1m_snapshot_forecast_	2026-07-11	1.0	149900000.0	-0.0517143	-0.0685306	0.007264210568402	no_buy	2026-08-11	no_positio	

v1				910552532	116065865	4			n	
vn_gold_20260711_3m_snapshot_forecast_v1	2026-07-11	3.0	149900000.0	0.027702584348732 7	-0.080230394745272 9	0.910857948679287 8	buy	2026-10-11	open	
vn_gold_20260711_5m_snapshot_forecast_v1	2026-07-11	5.0	149900000.0	0.096473853902500 9	-0.038352602084778 2	0.999999071415894 2	buy	2026-12-11	open	

Các bảng model và signal ở trên làm rõ khác biệt giữa dự báo, backtest signal và monitoring sau snapshot. Snapshot 3m/5m có tín hiệu buy, nhưng ledger vẫn open nên realized return chưa được tính; đây là điểm quan trọng để tránh đọc forecast như performance đã xảy ra.